



北京理工大学
Beijing Institute of Technology

本科实验报告

实验名称： 电磁理论、计算、应用 I 实验

课程名称：	电磁理论、计算、应用 I	实验时间：	2023 年 5 月
任课教师：	盛新庆、郭琨毅 吴比翼、杨明林	实验地点：	综教 B202
实验教师：	杨明林	实验类型：	☒ 原理验证 ☐ 综合设计 ☐ 自主创新
学生姓名：	王子赫		
学号/班级：	1120210446/13022101	组号：	无
学院：	集成电路与电子学院	同组搭档：	无
专业：	王小谟英才班	成绩：	



集成电路与电子学院
SCHOOL OF INTEGRATED CIRCUITS
AND ELECTRONICS

目录

实验一 一维平行板电容器电势的有限元方法求解	1
一、 实验目的	1
二、 实验原理	1
1、有限元方法	1
2、数学推导及编程原理	1
(1) 确定问题描述方程	1
(2) 确定泛函	1
(3) 区域离散	2
(4) 选取插值函数	2
(5) 建立方程组	3
(6) 矩阵组装	4
(7) 强加边界条件	5
(8) 求解矩阵方程	6
三、 实验过程	6
1、实验代码	6
2、代码解释	7
四、 实验结果	9
五、 实验总结	10
实验二 矩形空波导本征值问题的有限元方法求解	11
一、 实验目的	11
二、 实验原理	11
1、有限元方法	11
2、数学公式系统	11
(1) 矢量偏微分方程	11
(2) 推导泛函	12
①TE 模	12
②TM 模	13
(3) 基函数	13
(4) 泛函离散	14
(5) 广义本征方程求解	15
三、 实验程序编写	15
1、 实验程序编写思路及流程图	15

2、实验程序代码	16
(1) 主程序	16
(2) fill_matrix 函数	22
(3) derta 函数	25
(4) Triangle_area 函数	25
(5) TE_mode 函数	26
(6) TM_mode 函数	27
(7) add_boundary 函数	27
(8) Draw_TEMode 函数	28
(9) Draw_TMmode 函数	29
四、实验结果	30
1、特征值求解结果及其精度分析	30
(1) 网格划分	30
(2) 理论值计算	31
(3) 不同剖分下程序计算结果及与理论值的对比	33
2、特征值求解程序效率分析	34
(1) 尽量减少循环次数	35
(2) 矩阵组装时即算即装	35
(3) 强加边界条件采用对矩阵降维的方法	36
(4) 求解广义特征值采用稀疏矩阵	36
(5) 程序实际运行时间与效率分析	37
3、所解得的各模式下电场分布图	41
(1) TE 模	41
(2) TM 模	44
五、利用上述程序求解不规则波导的传播常数	48
1、特征值求解	49
(1) 网格划分	49
(2) 求解结果	49
2、程序运行时间与效率分析	50
3、绘制该不规则波导的场分布图像	52
(1) TE 模	52
(2) TM 模	54
六、实验总结	56

实验一 一维平行板电容器电势的有限元方法求解

一、实验目的

- 1、学习和掌握有限元方法。
- 2、学习和掌握 matlab 的使用方法。
- 3、利用 matlab 实现一维平行板电容器电势的有限元求解。

二、实验原理

1、有限元方法

有限元法是一种为求解偏微分方程边值问题近似解的数值技术。求解时对整个区域进行分解，每个子区域都成为简单的部分，这种简单部分就称作有限元。它通过变分方法，使得误差函数达到最小值并产生稳定解。

类比于连接多段微小直线逼近圆的思想，有限元法包含了一切可能的方法，这些方法将许多被称为有限元的小区域上的简单方程联系起来，并用其去估计更大区域上的复杂方程。它将求解域看成是由许多称为有限元的小的互连子域组成，对每一单元假定一个合适的（较简单的）近似解，然后推导求解这个域总的满足条件（如结构的平衡条件），从而得到问题的解。由于大多数实际问题难以得到准确解，而有限元不仅计算精度高，而且能适应各种复杂形状，因而成为行之有效的工程分析手段。

2、数学推导及编程原理

利用有限元法求解一维平行板电容器的电势，可以分为以下步骤：

（1）确定问题描述方程

该问题用可用泊松方程描述，简化为二阶微分方程：

$$\begin{aligned}\frac{d^2\Phi}{dx^2} &= x+1 & 0 < x < 1 \\ \Phi|_{x=0} &= 0 & \Phi|_{x=1} = 1\end{aligned}$$

（2）确定泛函

该问题的泛函为：

$$F(\Phi) = \frac{1}{2} \int_0^1 \left(\frac{d\Phi}{dx} \right)^2 dx + \int_0^1 (x+1)\Phi dx$$

(3) 区域离散

为了应用有限元方法，将区域离散为如图 1-1 的形式：

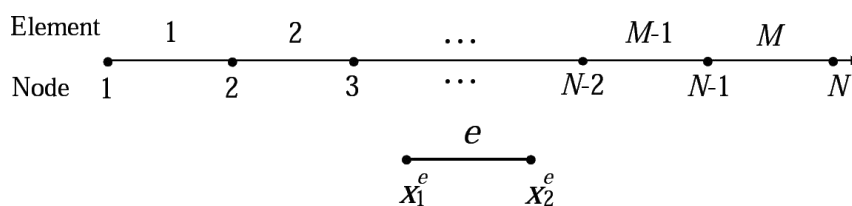


图 1-1 有限元方法的一维离散

图中上面表示将整个区域用 N 个点分解为 M 段，并对每一段和每个点都进行编号，即全局编号。

图中下面表示每一个被分割的小段，即小单元，左端标号 1 右端标号 2，即局部编号。

局部编号和全局编号之间的对应关系如表 1-1 所示：

表 1-1 一维离散局部编号和全局编号对应关系

e \ j		
	1	2
1	1	2
2	2	3
3	3	4
:	:	:
M	M	M+1

(4) 选取插值函数

根据离散方式，选择插值函数为：

$$\Phi^e = \Phi_i \frac{x_{i+1} - x}{x_{i+1} - x_i} + \Phi_{i+1} \frac{x - x_i}{x_{i+1} - x_i} \quad x_i \leq x \leq x_{i+1} \quad i = 1, 2, 3$$

$$= \sum_{j=1}^2 N_j^e \Phi_j^e = \begin{Bmatrix} \Phi_1^e & \Phi_2^e \end{Bmatrix} \begin{Bmatrix} N_1^e \\ N_2^e \end{Bmatrix} = \begin{Bmatrix} N_1^e & N_2^e \end{Bmatrix} \begin{Bmatrix} \Phi_1^e \\ \Phi_2^e \end{Bmatrix}$$

每个小区域的泛函为：

$$F_e = \frac{1}{2} \int_{x_1^e}^{x_2^e} \left(\frac{d\Phi^e}{dx} \right) \cdot \left(\frac{d\Phi^e}{dx} \right) dx + \int_{x_1^e}^{x_2^e} (x+1) \Phi^e dx$$

(5) 建立方程组

通过确定的插值函数可进一步推出：

$$\begin{aligned} \frac{\partial F^e}{\partial \Phi_1^e} &\rightarrow \frac{1}{2} \{1 \quad 0\} \int_{x_1^e}^{x_2^e} \begin{Bmatrix} \frac{dN_1^e}{dx} \\ \frac{dN_2^e}{dx} \end{Bmatrix} \cdot \left\{ \frac{dN_1^e}{dx} \quad \frac{dN_2^e}{dx} \right\} dx \begin{Bmatrix} \Phi_1^e \\ \Phi_2^e \end{Bmatrix} + \\ &\quad \frac{1}{2} \{ \Phi_1^e \quad \Phi_2^e \} \int_{x_1^e}^{x_2^e} \begin{Bmatrix} \frac{dN_1^e}{dx} \\ \frac{dN_2^e}{dx} \end{Bmatrix} \cdot \left\{ \frac{dN_1^e}{dx} \quad \frac{dN_2^e}{dx} \right\} dx \begin{Bmatrix} 1 \\ 0 \end{Bmatrix} \\ &= \int_{x_1^e}^{x_2^e} \frac{dN_1^e}{dx} \cdot \left\{ \frac{dN_1^e}{dx} \quad \frac{dN_2^e}{dx} \right\} dx \begin{Bmatrix} \Phi_1^e \\ \Phi_2^e \end{Bmatrix} \\ &= [K_{11}^e \quad K_{12}^e] \begin{Bmatrix} \Phi_1^e \\ \Phi_2^e \end{Bmatrix} \\ &\quad \{1 \quad 0\} \int_{x_1^e}^{x_2^e} (x+1) \begin{Bmatrix} N_1^e \\ N_2^e \end{Bmatrix} dx = \int_{x_1^e}^{x_2^e} (x+1) N_1^e dx \end{aligned}$$

同理可得：

$$\begin{aligned} \frac{\partial F^e}{\partial \Phi_2^e} &\rightarrow \int_{x_1^e}^{x_2^e} \frac{dN_2^e}{dx} \cdot \left\{ \frac{dN_1^e}{dx} \quad \frac{dN_2^e}{dx} \right\} dx \begin{Bmatrix} \Phi_1^e \\ \Phi_2^e \end{Bmatrix} \\ &= [K_{21}^e \quad K_{22}^e] \begin{Bmatrix} \Phi_1^e \\ \Phi_2^e \end{Bmatrix} \\ &\quad \{0 \quad 1\} \int_{x_1^e}^{x_2^e} (x+1) \begin{Bmatrix} N_1^e \\ N_2^e \end{Bmatrix} dx = \int_{x_1^e}^{x_2^e} (x+1) N_2^e dx \end{aligned}$$

归纳可得：

$$\begin{aligned} K_{ij}^e &= \int_{x_1^e}^{x_2^e} \frac{dN_1^e}{dx} \cdot \frac{dN_j^e}{dx} dx \\ b_i^e &= \int_{x_1^e}^{x_2^e} (x+1) N_i^e dx \end{aligned}$$

建立的方程为：

$$\begin{bmatrix} K_{11}^i & K_{12}^i \\ K_{21}^i & K_{22}^i \end{bmatrix} \begin{Bmatrix} \Phi_1^i \\ \Phi_2^i \end{Bmatrix} = \begin{Bmatrix} b_1^i \\ b_2^i \end{Bmatrix}$$

(6) 矩阵组装

以分成三段为例，阐述矩阵组装的原理。区域离散如图 1-2 所示：

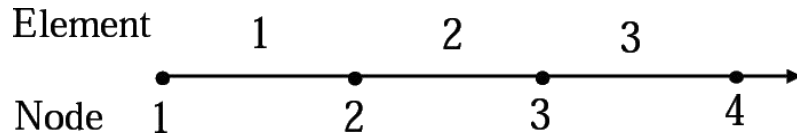


图 1-2 一维区域离散为三段

矩阵组装过程如图 1-3 所示：

$$\begin{aligned} K = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} + K^{(1)} = \begin{bmatrix} K_{11}^{(1)} & K_{12}^{(1)} \\ K_{21}^{(1)} & K_{22}^{(1)} \end{bmatrix} \Rightarrow K = \begin{bmatrix} K_{11}^{(1)} & K_{12}^{(1)} & 0 & 0 \\ K_{21}^{(1)} & K_{22}^{(1)} & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \\ \\ b = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} + b^{(1)} = \begin{bmatrix} b_1^{(1)} \\ b_2^{(1)} \end{bmatrix} \Rightarrow b = \begin{bmatrix} b_1^{(1)} \\ b_2^{(1)} \\ 0 \\ 0 \end{bmatrix} \\ \\ K = \begin{bmatrix} K_{11}^{(1)} & K_{12}^{(1)} & 0 & 0 \\ K_{21}^{(1)} & K_{22}^{(1)} & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} + K^{(2)} = \begin{bmatrix} K_{11}^{(2)} & K_{12}^{(2)} \\ K_{21}^{(2)} & K_{22}^{(2)} \end{bmatrix} \Rightarrow K = \begin{bmatrix} K_{11}^{(1)} & K_{12}^{(1)} & 0 & 0 \\ K_{21}^{(1)} & K_{22}^{(1)} + K_{11}^{(2)} & K_{12}^{(2)} & 0 \\ 0 & K_{21}^{(2)} & K_{22}^{(2)} & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \\ \\ b = \begin{bmatrix} b_1^{(1)} \\ b_2^{(1)} \\ 0 \\ 0 \end{bmatrix} + b^{(2)} = \begin{bmatrix} b_1^{(2)} \\ b_2^{(2)} \end{bmatrix} \Rightarrow b = \begin{bmatrix} b_1^{(1)} \\ b_2^{(1)} + b_1^{(2)} \\ b_2^{(2)} \\ 0 \end{bmatrix} \end{aligned}$$

图 1-3 矩阵组装过程示意图

最终组装好的矩阵为：

$$K = \begin{bmatrix} K_{11}^{(1)} & K_{12}^{(1)} & 0 & 0 \\ K_{21}^{(1)} & K_{22}^{(1)} + K_{11}^{(2)} & K_{12}^{(2)} & 0 \\ 0 & K_{21}^{(2)} & K_{22}^{(2)} + K_{11}^{(3)} & K_{12}^{(3)} \\ 0 & 0 & K_{21}^{(3)} & K_{22}^{(3)} \end{bmatrix} \quad b = \begin{bmatrix} b_1^{(1)} \\ b_2^{(1)} + b_1^{(2)} \\ b_2^{(2)} + b_1^{(3)} \\ b_2^{(3)} \end{bmatrix}$$

(7) 强加边界条件

该问题的边界条件为 $\Phi_1 = 0$ ， $\Phi_4 = 1$ ，需要强加在矩阵中。强加边界条件的方法和过程如图 1-4 所示：

$$K = \begin{bmatrix} K_{11} & K_{12} & K_{13} & K_{14} \\ K_{21} & K_{22} & K_{23} & K_{24} \\ K_{31} & K_{32} & K_{33} & K_{34} \\ K_{41} & K_{42} & K_{43} & K_{44} \end{bmatrix} \quad b = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \end{bmatrix} \quad \varphi_1 = p$$



$$K = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & K_{22} & K_{23} & K_{24} \\ 0 & K_{32} & K_{33} & K_{34} \\ 0 & K_{42} & K_{43} & K_{44} \end{bmatrix} \begin{bmatrix} \varphi_1 \\ \varphi_2 \\ \varphi_3 \\ \varphi_4 \end{bmatrix} = \begin{bmatrix} p \\ b_2 - K_{21} p \\ b_3 - K_{31} p \\ b_4 - K_{41} p \end{bmatrix}$$



$$\begin{bmatrix} K_{22} & K_{23} & K_{24} \\ K_{32} & K_{33} & K_{34} \\ K_{42} & K_{43} & K_{44} \end{bmatrix} \begin{bmatrix} \varphi_2 \\ \varphi_3 \\ \varphi_4 \end{bmatrix} = \begin{bmatrix} b_2 - K_{21} p \\ b_3 - K_{31} p \\ b_4 - K_{41} p \end{bmatrix}$$

图 1-4 强加边界条件过程示意图

(8) 求解矩阵方程

求解上述具有 $K\Phi = b$ 形式的矩阵方程组，得到的列向量 $\Phi = \begin{Bmatrix} \varphi_1 \\ \varphi_2 \\ \varphi_3 \\ \varphi_4 \end{Bmatrix}$ 即为

所求的解。将点 i 对应的 φ_i 用 **matlab** 绘出，可得到求解出的曲线，即利用有限元方法得到的该问题的数值解。

三、实验过程

1、实验代码

```
clear all;
% 求解区间[a,b]
a = 0;
b = 1;

% 将求解区间均匀地分成 n 个小段
n = 3;
h = (b - a) / n;

Q = a:h:b;           % 存全局坐标
K = zeros(n+1,n+1);  % 最后整个的大 κ 矩阵
B = zeros(n+1,1);    % 最后整个的 B 矩阵

% 算矩阵
syms x
for m = 1:n
    J = [Q(m),Q(m+1)]; % 局部坐标
    N = {@(x) (J(2)-x)/(J(2)-J(1)),@(x) (x-J(1))/(J(2)-J(1))}; % N1
    和 N2
    for i = 1:2
        for j = 1:2
            k(i,j) = int(diff(N{i},x)*...
                           diff(N{j},x),x,J(1),J(2)); %计算小矩阵 kij
            K(m+i-1,m+j-1) = K(m+i-1,m+j-1)+k(i,j); % 直接把小矩阵往大
        矩阵上加
    end
end
```

```

        b1(i,1) = int((x+1)*N{i},x,J(1),J(2)); % 计算 b
        B(m+i-1,1) = B(m+i-1,1)-b1(i); % 直接把小矩阵往大矩阵上加
    end
end

% 边界条件
B(1) = 0;
B(n+1) = 1;
K(1,:) = 0;
K(1,1) = 1;
K(n+1,:) = 0;
K(n+1,n+1) = 1;

S = linsolve(K,B); % 求解

% 绘制图像
plot(Q,S,'linewidth',2); % 绘制数值解
y = 1/6*x^3+0.5*x^2+1/3*x; % 精确解方程
hold on;
fplot(x,y,[0,1],'--','linewidth',2); % 绘精确值解
xlabel('x(m)'); % x 轴标签
ylabel('φ(V)'); % y 轴标签
ylim([0,1]); % y 轴范围
title('一维平行板电容器电势求解');
legend('数值解','精确解');

```

2、代码解释

在上述代码中，已经做了相当充足的注释，无需进行过多的细节上的解释，这里仅就代码编写思路和关键语句做出简要说明。

编写思路：

本代码编写思路与实验原理部分中的思路一模一样，没有改动，先对区域进行离散，再选取差值函数，按照推导出的数学公式对差值函数进行积分得到 K_{ij} 和 b_i ，根据局部编号和全局编号的对应关系将计算出的小矩阵 K_{ij} 和 b_i 加到一开始定义好的大矩阵 K 和 B 上， K 和 B 组装好后解矩阵方程 $K\Phi = B$ ，得到各个离散点的 φ_i 值，再用 `plot` 函数绘制出数值解曲线，同时也绘制出解析解的曲线用于对比。

为了节约内存和加快代码运行速度，在矩阵组装时采取了“即算即装、即装即走”的策略，即在算完每一个小区间后，就将小区间的结果组装到大矩阵上，不额外占用存储空间来存储每一个小区间的计算结果，每个小区间的数据组装完就覆盖掉。

关键语句说明：

```
N = { @(x) (J(2)-x)/(J(2)-J(1)), @(x) (x-J(1))/(J(2)-J(1)) }; % N1
和 N2
```

这句代码定义了数学推导中选取的两个插值函数 N_1 和 N_2 。

```
k(i,j) = int(diff(N{i},x)*...
              diff(N{j},x),x,J(1),J(2)); %计算小矩阵 kij
```

这句代码用于计算 K_{ij} 。

这句代码时是严格按照数学推导的表达式进行的，但事实上由于插值函数选择的是线性函数，求导简单，故该值可以手动算出，无需计算机进行积分，手动算出结果后可以对这句代码进行优化，提高程序的执行速度。

```
S = linsolve(K,B); % 求解
```

这句代码用于求解矩阵方程 $K\Phi = B$ ，得到 φ_i 。

```
y = 1/6*x^3+0.5*x^2+1/3*x; % 精确解方程
```

这句代码定义了该问题的解析表达式。

事实上，根据实验原理部分的数学推导，确定该问题的数学方程为：

$$\begin{aligned} \frac{d^2\Phi}{dx^2} &= x+1 & 0 < x < 1 \\ \Phi|_{x=0} &= 0 & \Phi|_{x=1} &= 1 \end{aligned}$$

这个方程是一个很简单的常微分方程，很容易求得解析解为：

$$\Phi(x) = \frac{1}{6}x^3 + \frac{1}{2}x^2 + \frac{1}{3}x$$

故将解析解和数值解在一个图窗中，便于对比数值解和理论解（解析解）的差异，以及划分区间的稀疏程度对求解结果准确度的影响。

四、实验结果

为对比区域的不同离散程度对解的准确性的影响，下面将列出上述代码运行的三个结果，分别是将求解区间 $[0,1]$ 划分为：

- ① $n = 3$;
- ② $n = 6$;
- ③ $n = 10$;

段时的结果，三个结果分别如图 1-5、1-6、1-7 所示：

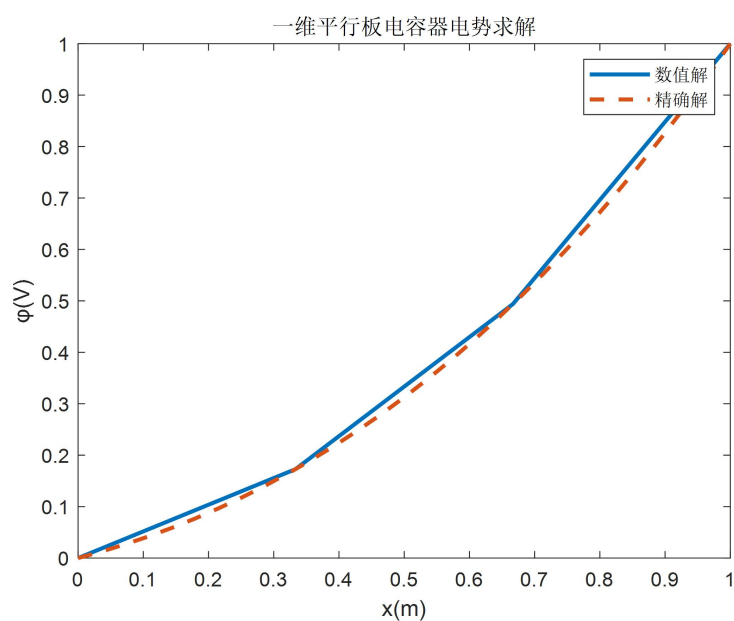


图 1-5 $n=3$ 时的求解结果

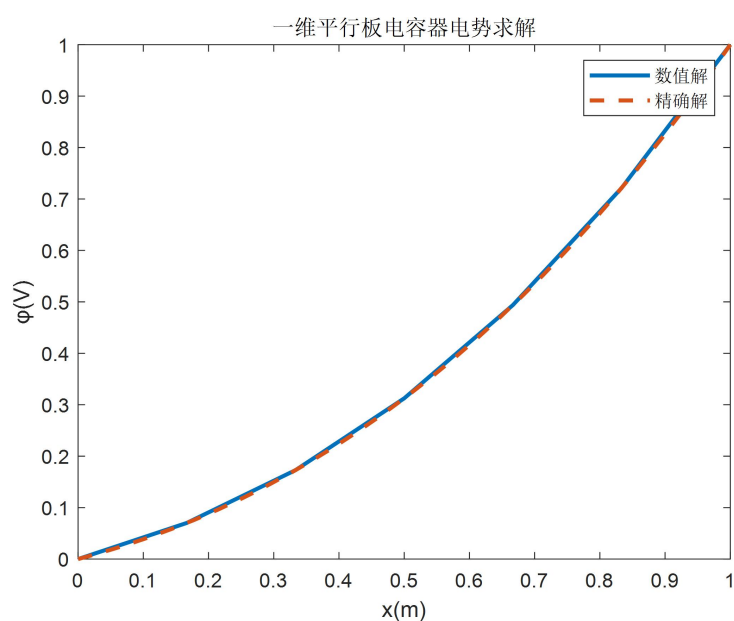


图 1-6 $n=6$ 时的求解结果

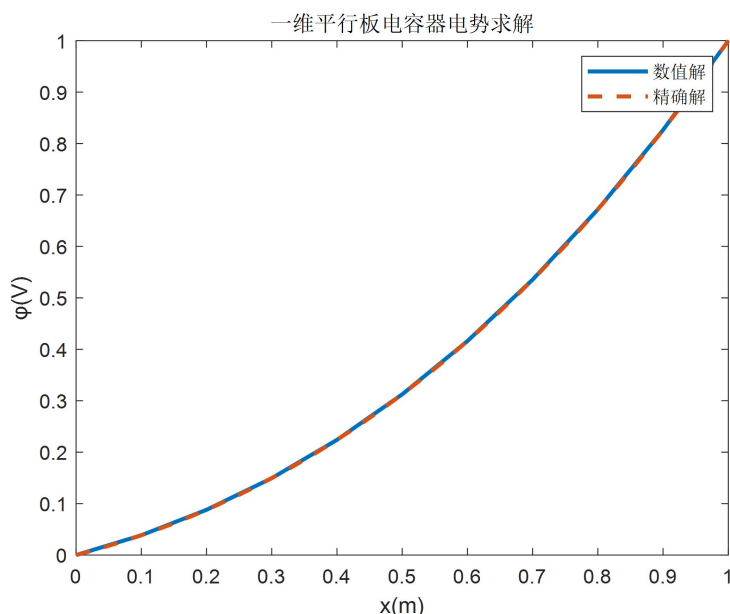


图 1-7 $n=10$ 时的求解结果

由图可知，随着离散程度的加深，实际值与理论值之间的差距越来越小，也说明该程序对一维平行板电容器电势的求解是正确的。

$n = 3$ 时，即使离散程度很小，但是也十分接近理论值曲线。当 $n = 10$ 时，几乎与理论值相一致。可想而知，继续增大离散的程度，增加 n 值，能够得到更加精确的结果，绘出更加精确的曲线。但是这时会导致程序运行时间的快速增长，考虑到 $n = 10$ 时结果已经相当准确，故 $n > 10$ 的离散程度实际上意义不大， $n = 10$ 的划分可以兼顾程序的效率与精度。

五、实验总结

通过本次一维平行板电容器电势的有限元方法求解实验，我初步学习和掌握有限元方法以及 matlab 的使用方法，利用 matlab 实现了一维平行板电容器电势的有限元求解。

本实验也是我第一次接触 matlab 代码，感受到了 matlab 工具在解决数学、工程方面问题的强大能力。

本实验用一维平行板电容器电势的有限元方法求解这样一个简单的例子，让我初步掌握了有限元方法，了解了有限元方法的由来、数学理论系统依据以及基本思路 and 实现方法。以解决这样一个简单的一维问题作为入门，对后续二维矩形空波导本征值问题打了牢固的基础，这样的课程安排循序渐进，由浅入深，由易入难，体验感十分良好。

实验二 矩形空波导本征值问题的有限元方法求解

一、实验目的

- 1、深入学习和掌握有限元方法。
- 2、深入学习和掌握 matlab 的使用方法。
- 3、利用 matlab 求解矩形空波导本征值问题。

二、实验原理

1、有限元方法

有限元法是一种为求解偏微分方程边值问题近似解的数值技术。求解时对整个问题区域进行分解，每个子区域都成为简单的部分，这种简单部分就称作有限元。它通过变分方法，使得误差函数达到最小值并产生稳定解。

类比于连接多段微小直线逼近圆的思想，有限元法包含了一切可能的方法，这些方法将许多被称为有限元的小区域上的简单方程联系起来，并用其去估计更大区域上的复杂方程。它将求解域看成是由许多称为有限元的小的互连子域组成，对每一单元假定一个合适的（较简单的）近似解，然后推导求解这个域总的满足条件（如结构的平衡条件），从而得到问题的解。由于大多数实际问题难以得到准确解，而有限元不仅计算精度高，而且能适应各种复杂形状，因而成为行之有效的工程分析手段。

2、数学公式系统

利用有限元法求解矩形空波导本征值问题所用数学公式系统如下：

（1）矢量偏微分方程

求解波导问题的一般途径是先解出纵向场，后由纵向场直接求出横向场。

柱坐标系下，电场和磁场的标量亥姆霍兹方程如下：

$$\begin{aligned}(\nabla^2 + k^2)E_z &= 0 \\ (\nabla^2 + k^2)H_z &= 0\end{aligned}$$

电磁波在波导中传输，横向上呈固定场分布，纵向上以一定速度传播：

$$\begin{aligned}\vec{E}(\vec{r}) &= E(t)e^{-\gamma z} \\ \vec{H}(\vec{r}) &= H(t)e^{-\gamma z} \\ k_c &= k^2 + \gamma^2\end{aligned}$$

最终得到：

$$\nabla_t^2 E_z + k_c^2 E_z = 0 \quad TM \text{模}$$

$$\nabla_t^2 H_z + k_c^2 H_z = 0 \quad TE \text{模}$$

该问题的二维模型如图 2-1 所示：

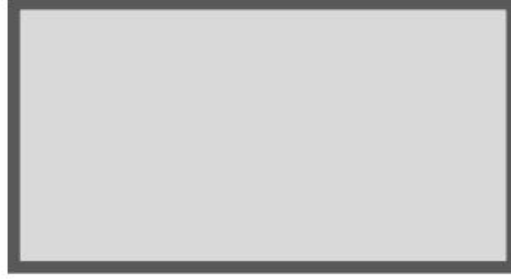


图 2-1 波导本征值问题二维模型

(2) 推导泛函

①TE 模

$$\nabla_t^2 H_z + k_c^2 H_z = 0$$

在边界上

$$\left. \frac{\partial H_z}{\partial n} \right|_{\Gamma} = 0$$

两端同乘 $\delta(H_z)$ 并做积分：

$$\int_S (\nabla^2 H_z + k_c^2 H_z) \delta H_z dS - \int_{\Gamma} \frac{\partial H_z}{\partial n} \cdot \delta H_z dl = 0$$

由第一类标量格林定理

$$\int_S \delta H_z \cdot \nabla^2 H_z dS + \int_S \nabla \delta H_z \cdot \nabla H_z dS = \int_{\Gamma} \frac{\partial H_z}{\partial n} \cdot \delta H_z dl$$

可推出泛函为：

$$F(H_z) = \frac{1}{2} \int_S (\nabla H_z \cdot \nabla H_z - k_c^2 H_z^2) dS$$

②TM 模

$$\nabla_t^2 E_z + k_c^2 E_z = 0$$

在边界上

$$E_z = 0$$

两端同乘 $\delta(E_z)$ 并做积分式相加得：

$$\int_S (\nabla_t^2 E_z + k_c^2 E_z) \delta E_z dS = 0$$

由第一类标量格林定理

$$\int_S \delta E_z \cdot \nabla_t^2 E dS + \int_S \nabla_t \delta E_z \cdot \nabla_t E dS = \int_{\Gamma} \frac{\partial E_z}{\partial n} \cdot \delta E_z dl$$

可推出泛函为：

$$F(E_z) = \frac{1}{2} \int_S (\nabla_t E_z \cdot \nabla_t E_z - k_c^2 E_z^2) dS$$

(3) 基函数

$$L_i^e(x, y) = \frac{1}{2\Delta^e} (a_i^e + b_i^e x + c_i^e y) = \frac{\Delta_i^e}{\Delta}$$

$$a_1^e = x_2^e y_3^e - x_3^e y_2^e; \quad b_1^e = y_2^e - y_3^e; \quad c_1^e = x_3^e - x_2^e;$$

$$a_2^e = x_3^e y_1^e - x_1^e y_3^e; \quad b_2^e = y_3^e - y_1^e; \quad c_2^e = x_1^e - x_3^e;$$

$$a_3^e = x_1^e y_2^e - x_2^e y_1^e; \quad b_3^e = y_1^e - y_2^e; \quad c_3^e = x_2^e - x_1^e;$$

顶点元基函数定义示意图如图 2-2 所示：

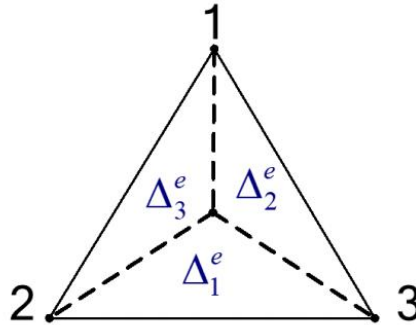


图 2-2 顶点元基函数定义示意图

基函数数值变化情况如图 2-3 所示：

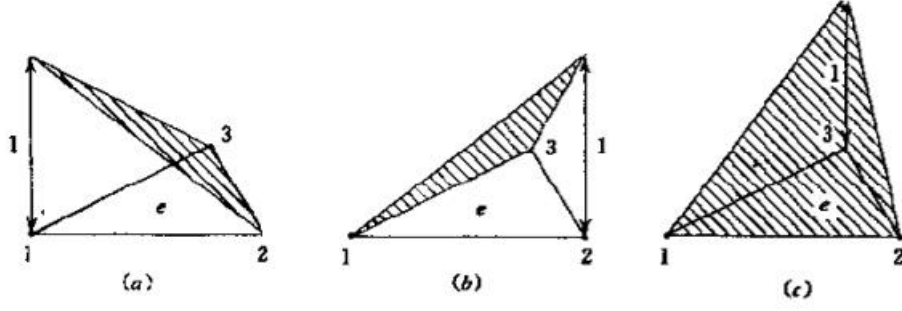


图 2-3 顶点元基函数数值变化情况

(4) 泛函离散

已知泛函：

$$F(E_z) = \frac{1}{2} \int_S (\nabla_t E_z \cdot \nabla_t E_z - k_c^2 E_z^2) dS$$

在单元内展开：

$$F = \frac{1}{2} \sum_{e=1}^M \left(\{E_z^e\}^T [A^e] \{E_z^e\} - k_c^2 \{E_z^e\}^T [B^e] \{E_z^e\} \right)$$

$$[A^e] = \int_S (\nabla L^e \cdot \nabla \{L^e\}^T) dS$$

$$[B^e] = \int_S (L^e \cdot \{L^e\}^T) dS$$

求变分得：

$$[A] \{E_z\} = k_c^2 [B] \{E_z\}$$

其中

$$A = \sum_{e=1}^M [A^e]$$

$$B = \sum_{e=1}^M [B^e]$$

$$A_{ij} = \frac{1}{4\Delta} (b_i b_j + c_i c_j) \quad B_{ij} = \frac{1 + \delta_{ij}}{12} \Delta$$

这组解析方程也是后续编程过程中的重要依据公式。

(5) 广义本征方程求解

组装好矩阵后，需求解的方程为：

$$[A]\{E_z\} = k_c^2 [B]\{E_z\}$$

三、实验程序编写

1、实验程序编写思路及流程图

在实验原理部分已经确定了所需的公式系统，其大部分内容是公式的推导，对程序编写起到关键作用的只有最终的结论。

根据推导出的问题求解公式，可以确定如下程序编写思路：

- (1) 用 **pde** 工具对求解区域进行网格划分
- (2) 导出划分后形成的 **p**、**e**、**t** 矩阵
- (3) 根据公式系统得出的最终解析结果进行矩阵的填充（最难实现）
- (4) 求解 **TE** 模式本征值
- (5) 添加边界条件
- (6) 求解 **TM** 模式本征值
- (7) 绘制并观察场的分布图像

根据该思路，可以绘制出程序编写的流程图如图 3-1 所示：



图 3-1 有限元法求解矩形空波导本征值问题程序编写流程图

2、实验程序代码

(1) 主程序

```
clear;

% 加载网格
% load('waveguide_1_mesh1');
% load('waveguide_1_mesh2');
load('waveguide_1_mesh3');
% load('waveguide_2_mesh');

% 填充矩阵
[A,B]= fill_matrix(p,t);

% 求 TE 特征值
num_TE = 10;
[Ez_TE,kc_TE]= TE_mode(A,B,num_TE);
disp(kc_TE)

% 求 TM 特征值
num_TM = 10;
[Ez_TM,kc_TM]= TM_mode(A,B,e,num_TM);
disp(kc_TM)

% 绘制场模式图
mode_TE = 1;
mode_TM = 1;
Draw_TEMode(t,p,Ez_TE,mode_TE); % 绘制 TE 模式场分布图
figure();
Draw_TMmode(t,p,e,Ez_TM,mode_TM); % 绘制 TM 模式场分布图
```

主程序的编写完全按照流程图的顺序，各个函数的含义，程序的功能，以及各个变量的含义通过命名和注释均可知道，不做过多介绍。

唯独需要介绍的是加载网格的部分。在主程序文件夹中，网格划分文件一共四组，如图 3-2 所示：

 waveguide_1_mesh1.m	2023/5/21 2:22	M 文件	2 KB
 waveguide_1_mesh1.mat	2023/5/21 8:36	MAT 文件	3 KB
 waveguide_1_mesh2.m	2023/5/21 2:23	M 文件	2 KB
 waveguide_1_mesh2.mat	2023/5/21 8:37	MAT 文件	47 KB
 waveguide_1_mesh3.m	2023/5/21 2:23	M 文件	2 KB
 waveguide_1_mesh3.mat	2023/5/21 8:38	MAT 文件	190 KB
 waveguide_2_mesh.m	2023/5/6 17:26	M 文件	3 KB
 waveguide_2_mesh.mat	2023/5/21 8:39	MAT 文件	183 KB

图 3-2 网格划分文件

其中 waveguide_1_mesh 系列共有 3 组，分别为

waveguide_1_mesh1

waveguide_1_mesh2

waveguide_1_mesh3

该系列网格划分对应的波导形状如图 3-3 所示：

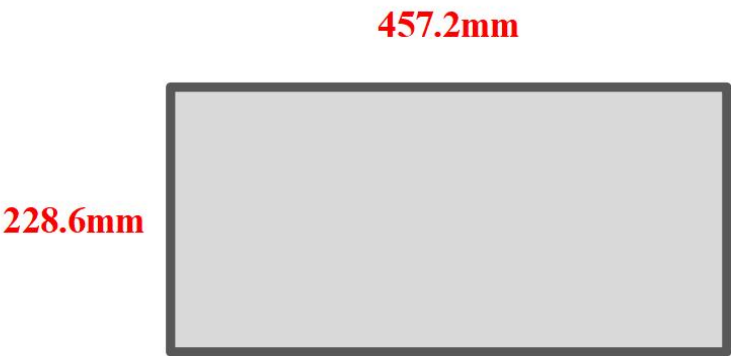


图 3-3 waveguide_1_mesh 系列对应的波导形状

从 mesh1 到 mesh3，均是对该波导的网格划分，只不过划分疏密程度不一样。waveguide_1_mesh1 划分最稀疏，waveguide_1_mesh3 最密集。

以.m 后缀结尾的是可执行文件，比如用 matlab 打开 waveguide_1_mesh1.m 文件并点击运行，可以绘出对该波导的划分，这时可以通过点击加载来将生成的 p、e、t 矩阵导入工作区。

p、e、t 三个矩阵保存了使用该网格的全部信息，下面将分别给出三个矩阵的存储形式并说明其存储的内容。

p 矩阵的矩阵形式如图 3-4 所示：

p											
2x7347 double											
	1	2	3	4	5	6	7	8	9	10	11
1	0	0.4572	0.4572	0	0.0050	0.0099	0.0149	0.0199	0.0248	0.0298	0.0348
2	0.2286	0.2286	0	0	0.2286	0.2286	0.2286	0.2286	0.2286	0.2286	0.2286

图 3-4 p 矩阵形式

p 矩阵存储的是各个点的坐标。比如第 6 列的两个数据，第六列第一行的数据就是全局编号为 6 的点的横坐标，第六列第二行的数据就是全局编号为 6 的点的纵坐标。

e 矩阵的矩阵形式如图 3-5 所示：

e											
7x276 double											
	1	2	3	4	5	6	7	8	9	10	11
1	1	5	6	7	8	9	10	11	12	13	14
2	5	6	7	8	9	10	11	12	13	14	15
3	0	0.0109	0.0217	0.0326	0.0435	0.0543	0.0652	0.0761	0.0870	0.0978	0.1087
4	0.0109	0.0217	0.0326	0.0435	0.0543	0.0652	0.0761	0.0870	0.0978	0.1087	0.1196
5	1	1	1	1	1	1	1	1	1	1	1
6	0	0	0	0	0	0	0	0	0	0	0
7	1	1	1	1	1	1	1	1	1	1	1

图 3-5 e 矩阵形式

e 矩阵在本程序中用的是第一行，存储了所有边界上点的全局编号，将在求解 TM 模式，强加边界条件的时候用于剔除边界上的点。

t 矩阵的矩阵形式如图 3-6 所示：

t											
4x14416 double											
	1	2	3	4	5	6	7	8	9	10	11
1	5	6	7	8	9	10	11	12	13	14	15
2	1	5	6	7	8	9	10	11	12	13	14
3	365	360	308	358	891	1289	1244	834	601	1278	903
4	1	1	1	1	1	1	1	1	1	1	1

图 3-6 t 矩阵形式

t 矩阵的前三行储存了各个三角形对应的三个点的全局编号。比如第六列的前三行存储的分别是全局编号为 6 的小三角形的第一个点、第二个点、第三个点的全局编号。

为了更加方便的运行程序与解决问题，已经将各个网格生成的 p、e、t 矩阵导出，并形成了后缀为.mat 的文件。

其中，`waveguide_1_mesh1.m` 文件生成的网格导出的 p 、 e 、 t 矩阵，就保存在文件 `waveguide_1_mesh1.mat` 里，其他以此类推。

`waveguide_1_mesh1.m` 文件生成的网格如图 3-7 所示：

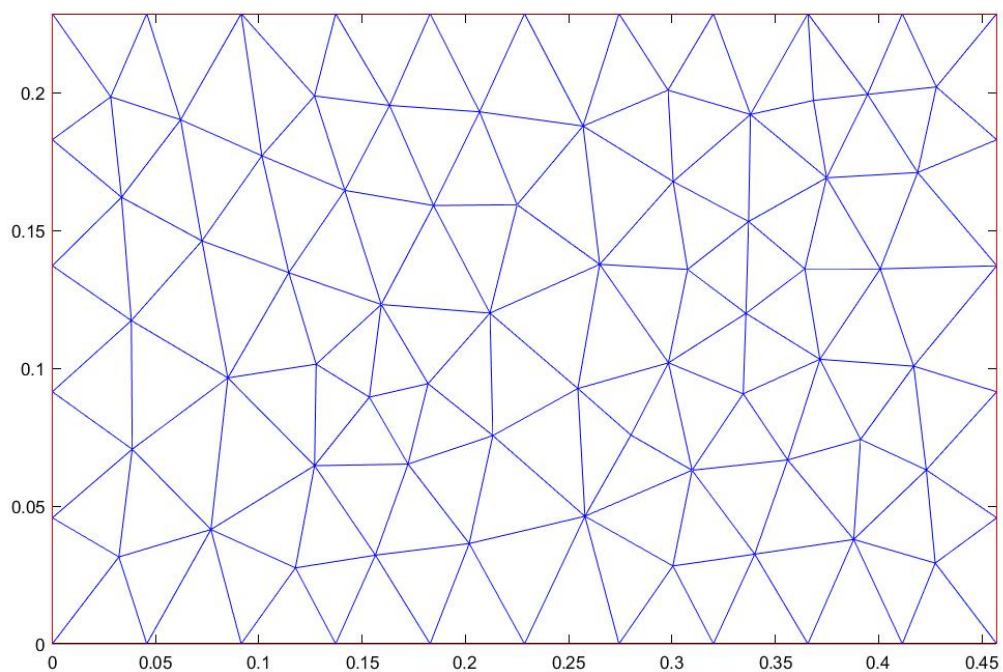


图 3-7 `waveguide_1_mesh1.m` 文件生成的网格

`waveguide_1_mesh2.m` 文件生成的网格如图 3-8 所示：

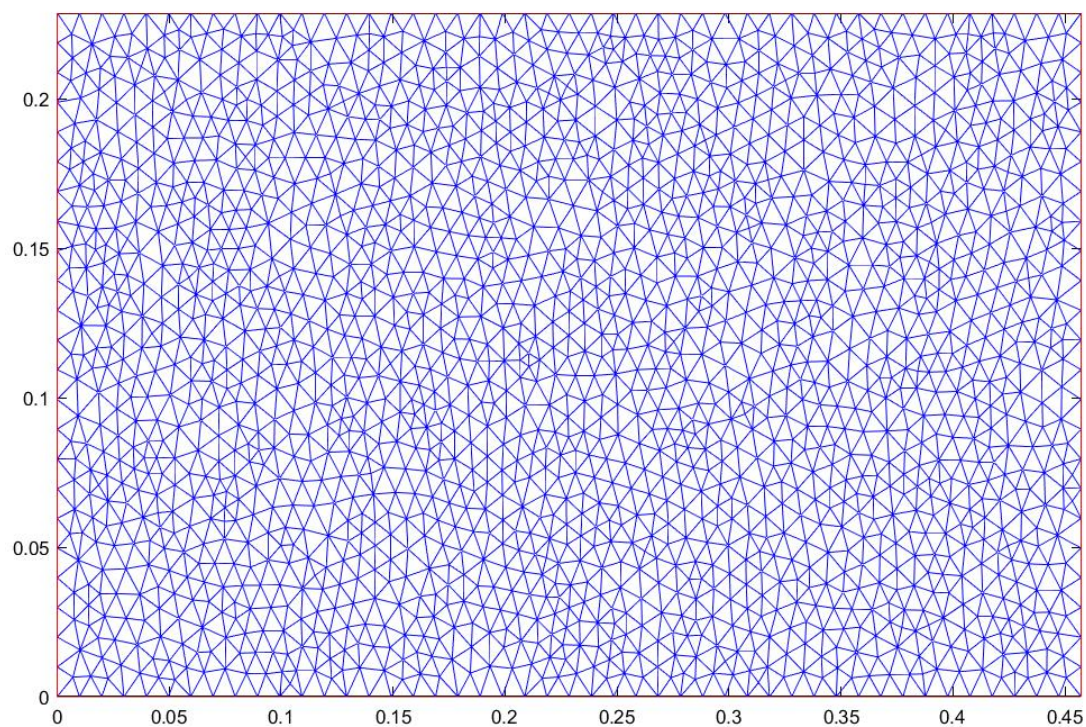


图 3-8 `waveguide_1_mesh2.m` 文件生成的网格

waveguide_1_mesh3.m 文件生成的网格如图 3-9 所示：

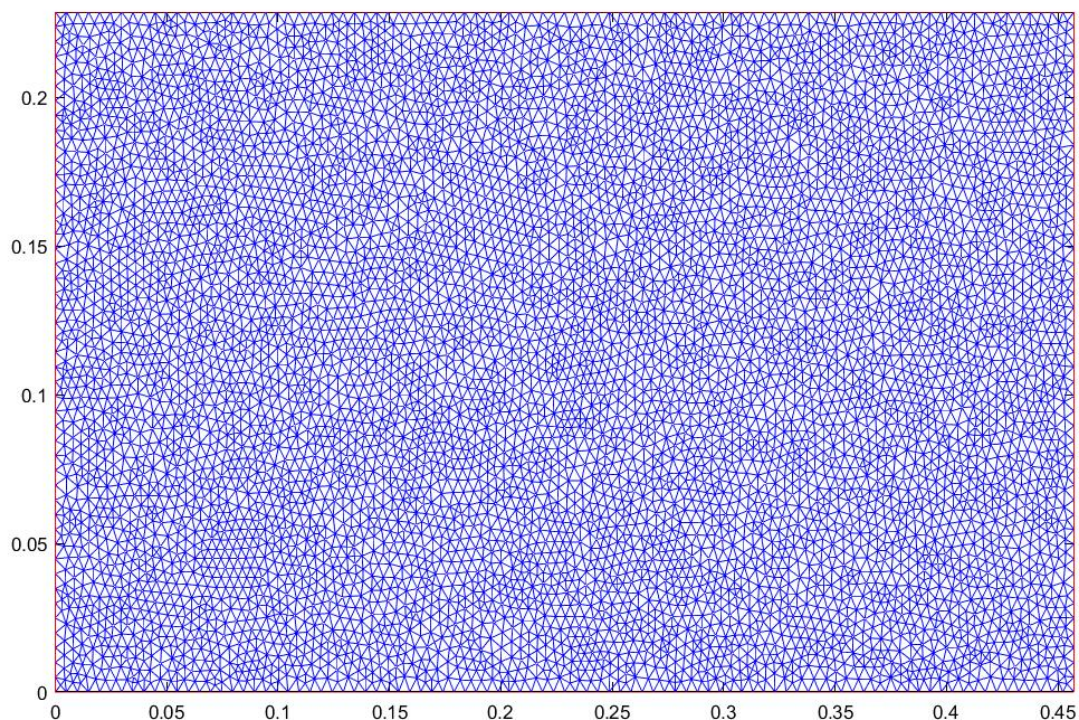


图 3-9 waveguide_1_mesh3.m 文件生成的网格

最后一组是 waveguide_2_mesh，该系列只有这一组网格，并且网格划分较细。该组网格划分对应的波导形状如图 3-10 所示：

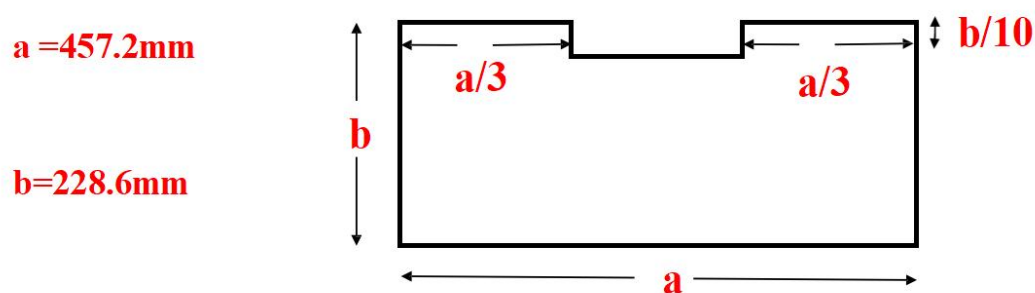


图 3-10 waveguide_2_mesh 系列对应的波导形状

以.m 后缀结尾的是可执行文件，比如用 matlab 打开 waveguide_2_mesh.m 文件并点击运行，可以绘出对该波导的划分，这时可以通过点击加载来将生成的 p、e、t 矩阵导入工作区。

同样，waveguide_2_mesh.m 文件生成的网格导出的 p、e、t 矩阵，就保存在文件 waveguide_2_mesh.mat 里。

waveguide_2_mesh.m 文件生成的网格如图 3-11 所示：

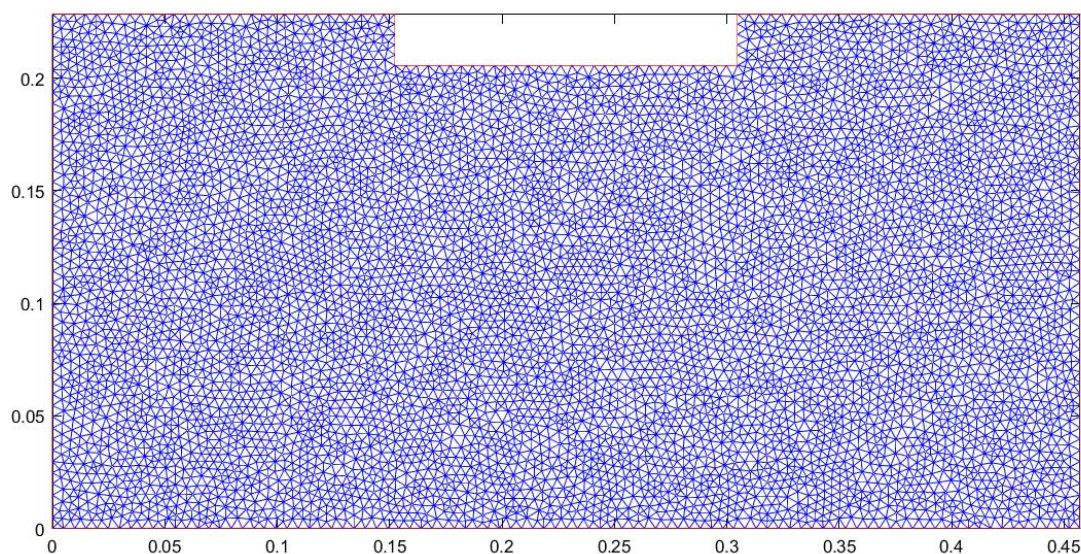


图 3-11 waveguide_2_mesh.m 文件生成的网格

在解决问题与运行程序时，只需要将.mat 文件里的矩阵导入到工作区即可运行，如主程序开头加载网格部分所示：

```
% 加载网格  
  
% load('waveguide_1_mesh1');  
  
% load('waveguide_1_mesh2');  
  
load('waveguide_1_mesh3');  
  
% load('waveguide_2_mesh');
```

图 3-12 主程序开头加载网格部分

由于文件夹中共生成了四个文件，所以主程序里将这四个网格的划分都写了进去，并用注释来决定到底加载哪个网格，解决哪个波导问题。

load 函数加载的文件实际上是.mat 文件，加载后网格划分的 p 、 e 、 t 矩阵直接就加载到了工作区中，后续函数可以使用并运行，得到矩形空波导本征值问题的解。

当然，也可以将这四句话都注释掉，然后绘制新的网格加载到工作区进行运行，这时务必注意也要将前面的 clear 注释掉，否则刚加载的矩阵就被清理掉了。另外每重新加载一个网格之前，为了避免意外，都需要手动清理工作区。

下面将着重按各个函数在主函数中出现的顺序逐个展示，并介绍各个函数编写思想和功能，更好地展现程序执行的流程。

(2) fill_matrix 函数

% 算矩阵

```
function [A,B]= fill_matrix(p,t)
```

```
n = size(p,2);
```

```
A = zeros(n,n);
```

```
B = zeros(n,n);
```

```
for m = 1:size(t,2)
```

```
    x = p(1,t(1:3,m)); % 计算小三角形顶点的全局坐标
```

```
    y = p(2,t(1:3,m));
```

```
    L = [2,3;3,1;1,2];
```

% 计算小三角形面积

```
s = Triangle_area(x,y);
```

% 矩阵组装

```
for i = 1:3
```

```
    bi = y(L(i,1))-y(L(i,2));
```

```
    ci = x(L(i,2))-x(L(i,1));
```

```
    for j = 1:3
```

```
        bj = y(L(j,1))-y(L(j,2));
```

```
        cj = x(L(j,2))-x(L(j,1));
```

% 填充 A 矩阵

```
A(t(i,m),t(j,m)) = A(t(i,m),t(j,m)) + (bi*bj+ci*cj)/(4*s);
```

% 填充 B 矩阵

```

        B(t(i,m),t(j,m)) = B(t(i,m),t(j,m)) + (1+derta(i,j))*s/12;

    end

end

end

return

```

fill_matrix 函数主要功能为填充 A, B 矩阵, 为后续解广义特征方程做准备。
该函数是代码中最重要, 规模最大、最难实现的函数。

组装时用到的公式为:

$$\begin{aligned}
 b_1^e &= y_2^e - y_3^e; & c_1^e &= x_3^e - x_2^e; \\
 b_2^e &= y_3^e - y_1^e; & c_2^e &= x_1^e - x_3^e; \\
 b_3^e &= y_1^e - y_2^e; & c_3^e &= x_2^e - x_1^e;
 \end{aligned}$$

$$A = \sum_{e=1}^M [A^e]$$

$$B = \sum_{e=1}^M [B^e]$$

$$A_{ij} = \frac{1}{4\Delta} (b_i b_j + c_i c_j) \quad B_{ij} = \frac{1 + \delta_{ij}}{12} \Delta$$

下面讨论一下矩阵组装时的具体细节。

fill_matrix 函数填充矩阵的原理很简单, 即根据每个小三角形计算出的 3×3 小矩阵对应的全局编号的位置填充整个大的矩阵, 下面将举例说明。

比如一个三角形计算出的 Am 是一个 3×3 矩阵, 那么这个矩阵的第 (1,3) 个位置的值就应该组装到 A 矩阵的 (t(1,m), t(3,m)) 的位置, 即小三角形局部编号为 1 和局部编号为 3 的点确定的全局编号确定的位置。

为了更加方便的计算 bi, bj, ci, cj, 这里构造一个矩阵 L = [2,3;3,1;1,2], 通过 L 加循环的控制就可以较为简单的计算出上述变量。

另外, fill_matrix 采用的即算即装模式, 提高了效率, 将在后续分析。

fill_matrix 函数中用到的 derta 函数, 也将在后续说明。

A 矩阵的形式如图 3-13 所示：

A								
7347x7347 double								
	1	2	3	4	5	6	7	8
1	1.0832	0	0	0	0.0309	0	0	0
2	0	0.8550	0	0	0	0	0	0
3	0	0	0.8683	0	0	0	0	0
4	0	0	0	1.0873	0	0	0	0
5	0.0309	0	0	0	1.7948	-0.2422	0	0
6	0	0	0	0	-0.2422	1.7517	-0.2562	0
7	0	0	0	0	0	-0.2562	1.7384	-0.2317
8	0	0	0	0	0	0	-0.2317	1.7580
9	0	0	0	0	0	0	0	-0.1845
10	0	0	0	0	0	0	0	0
11	0	0	0	0	0	0	0	0
12	0	0	0	0	0	0	0	0
13	0	0	0	0	0	0	0	0
14	0	0	0	0	0	0	0	0
15	0	0	0	0	0	0	0	0
16	0	0	0	0	0	0	0	0

图 3-13 A 矩阵形式

B 矩阵的形式如图 3-14 所示：

B										
7347x7347 double										
	4	5	6	7	8	9	10	11	12	13
1	0	4.8205e-07	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0
4	1.9036e-06	0	0	0	0	0	0	0	0	0
5	0	4.7652e-06	8.0848e-07	0	0	0	0	0	0	0
6	0	8.0848e-07	4.8243e-06	8.3979e-07	0	0	0	0	0	0
7	0	0	8.3979e-07	4.9138e-06	8.0455e-07	0	0	0	0	0
8	0	0	0	8.0455e-07	4.4936e-06	7.3582e-07	0	0	0	0
9	0	0	0	0	7.3582e-07	4.2825e-06	7.7950e-07	0	0	0
10	0	0	0	0	0	7.7950e-07	6.1906e-06	7.3073e-07	0	0
11	0	0	0	0	0	0	7.3073e-07	4.4274e-06	8.4979e-07	0
12	0	0	0	0	0	0	0	8.4979e-07	5.4356e-06	9.5217e-07
13	0	0	0	0	0	0	0	0	9.5217e-07	5.4193e-06
14	0	0	0	0	0	0	0	0	0	7.6221e-07
15	0	0	0	0	0	0	0	0	0	0
16	0	0	0	0	0	0	0	0	0	0

图 3-14 B 矩阵形式

可以看出，两个矩阵均是对称的、稀疏的，符合理论分析，也能够证明该程序组装的矩阵是正确的。

(3) derta 函数

% 相同为 1, 不同为 0

```
function s = derta(i,j)
```

```
    if(i==j)
```

```
        s = 1;
```

```
    else
```

```
        s = 0;
```

```
    end
```

```
return
```

derta 函数用于判断两个数是否相同，相同为 1，不同为 0。

derta 函数用于矩阵组装过程中 **B** 的计算，利用该函数的公式如下：

$$B_{ij} = \frac{1 + \delta_{ij}}{12} \Delta$$

(4) Triangle_area 函数

```
function s = Triangle_area(x,y)
```

```
% 计算三角形面积
```

```
s = 0.5 * abs((x(1)-x(3))*(y(2)-y(3)) - (x(2)-x(3))*(y(1)-y(3)));
```

```
return
```

Triangle_area 函数根据三角形面积计算公式，计算三角形的面积并返回。输入参数为三角形的三个点的坐标，返回值为三角形面积。主要用于矩阵填充过程中小三角形面积的计算，利用到该函数的公式如下：

$$A_{ij} = \frac{1}{4\Delta} (b_i b_j + c_i c_j) \quad B_{ij} = \frac{1 + \delta_{ij}}{12} \Delta$$

(5) TE_mode 函数

```
% 求 TE 特征值

function [Ez, kc]= TE_mode(A,B,num)

% 转为稀疏矩阵提高运算速度

A = sparse(A);
B = sparse(B);

[Ez,D]= eigs(A,B,num+1,'smallestabs');
kc = abs(diag(D).^0.5);           % 提取出特征值

% 第一个为 0，删去
kc(1) = [];
Ez(:,1) = [];

return
```

TE_mode 函数用于解该空波导的 TE 模式本征值。所用数学公式为：

$$[A]\{E_z\} = k_c^2 [B]\{E_z\}$$

经过 fill_matrix 函数，A 与 B 矩阵均已组装好，只要求矩阵方程的广义特征方程，得到特征值，开根即可解决问题。Ez 是该特征方程的特征向量，存储各个点处的数值，用于后续绘制电场的图像。

TE_mode 还有一个输入参数 num。该参数可以在主程序中传进去，用于确定解前多少个本征值。比如 num = 10，那就只需要解出前 10 个本征值即可，这样可以提高效率，并且使得程序更加灵活，不浪费资源。

由于 TE 模式下第一个特征值为 0，故将这个特征值与其对应的特征向量删去，防止干扰。

同时也由于删去了一个为 0 的特征值，在求解特征值的时候在数量上需要再增加一个，才能满足要求。比如要求计算前 10 个本征值，实际计算时需要计算 11 个，即 num+1，才能在删去第一个后满足 10 个本征值的条件。

（6）TM_mode 函数

```
% 求 TM 特征值
function [Ez,kc]= TM_mode(A,B,e,num)

    [A,B] = add_boundary(A,B,e);

    % 转为稀疏矩阵提高运算速度
    A = sparse(A);
    B = sparse(B);

    [Ez,D]= eigs(A,B,num,'smallestabs');
    kc = abs(diag(D).^0.5);           % 提取出特征值

return
```

TM_mode 函数用于解该空波导的 TM 模式本征值。其编写思路与理论依据均与 TE_mode 函数相同，这里不再赘述。

唯一与 TE 模式的解不同的地方是，TM 模式在解之前需要添加边界条件，添加边界条件的函数将会在下面介绍。

另外，TM 模式没有为 0 的特征值，所以不用删除第一个特征值，求 num 个特征值只需要计算 num 个就可以完成。

（7）add_boundary 函数

```
% 添加边界条件，删去矩阵中处于边界处的行和列
function [A,B] = add_boundary(A,B,e)

    Nnode = size(A);

    inner_node_num = 1:Nnode;
    inner_node_num(e(1,:)) = [];

    A = A(inner_node_num,inner_node_num);
    B = B(inner_node_num,inner_node_num);

return
```

add_boundary 函数用于强加边界条件。

由于在 PEC 边界上，TM 模式下， $E_z=0$ ，故需要对组装好的矩阵强加边界条件，较为简单的方法就是对矩阵进行降维，将在边界上的边或点确定的行和列删除即可。

`add_boundary` 函数根据 `e` 矩阵储存的边界上点的全局编号，将编号所对应的行和列删除，一方面强加了边界条件（因为已经知道这些点出解出来的值应该为 0），另一方面矩阵也实现了降维，提高了效率。

（8）Draw_TEmode 函数

% 绘制 TE 模式的传输图像

```
function []= Draw_TEmode(t,p,Ez,mode)
```

```
% 绘制 Ez
```

```
trisurf(t(1:3,:),p(1,:),p(2,:),0*ones(size(p(2,:))),...
```

```
abs(Ez(:,mode)), 'facecolor','interp','edgecolor','none');
```

```
axis equal
```

```
colormap jet
```

```
colorbar
```

```
title('TE mode')
```

```
hold on
```

```
% 平面问题，更改视角，方便观察
```

```
view(0,90);
```

```
return
```

`Draw_TEmode` 函数用于绘制 TE 模式下场的分布，函数可通过参数 `mode` 决定绘制的模式。

广义特征方程已经求出的特征值和特征向量，特征向量是各个点处 E_z 的值，并且由于 TE 模式不涉及强加边界条件删除边界上的点的问题，故 E_z 是完备的，可以直接绘图。

绘图时，将 E_z 作为颜色值参数，用 `trisurf` 函数绘制出电场分布彩图，图的颜色代表 E_z 的大小。

(9) Draw_TMmode 函数

```
% 绘制 TM 模式的传输图像

function []= Draw_TMmode(t,p,e,Ez,mode)

% 补全 Ez

inner_node = 1:size(p,2);

inner_node(e(1,:)) = [];

Ez0(inner_node) = Ez(:,mode);

Ez0(e(1,:)) = 0;

% 绘制 Ez

trisurf(t(1:3,:),p(1,:),p(2,:),0*ones(size(p(2,:))),...
        abs(Ez0),'facecolor','interp','edgecolor','none');

axis equal

colormap jet

colorbar

title('TM mode')

hold on

% 平面问题，更改视角，方便观察

view(0,90);

return
```

Draw_TMmode 函数用于绘制 TM 模式下场的分布，函数可通过参数 mode 决定绘制的模式。

Draw_TMmode 函数的编写思路与绘图原理均与 Draw_TEmode 函数一致，这里不再赘述。

与 Draw_TEmode 函数唯一不同的是，由于解 TM 模式需要强加边界条件，而由于强加边界条件采取的删除矩阵的方法，这时的 Ez 是不完备的，缺少边界上的点的值，而这些值是 0，只需要通过 e 矩阵存储的边界上的点的全局编号的信息，将这些 0 重新组装回去，得到完备的 Ez 矩阵即可。

四、实验结果

1、特征值求解结果及其精度分析

(1) 网格划分

如介绍主程序开头加载网格部分那样，为了解该问题，一共提前生成了三个该空波导的网格划分。为更加直观感受到划分密集程度以及其对求解结果的影响，现将三种划分的图像结果再次在这里绘出。

网格 1 划分如图 4-1 所示：

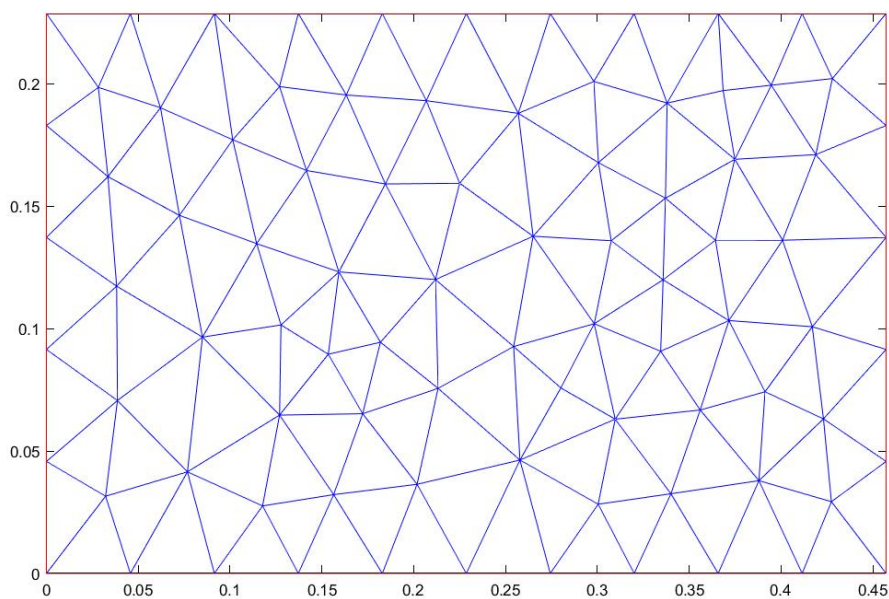


图 4-1 网格 1 划分

网格 2 划分如图 4-2 所示：

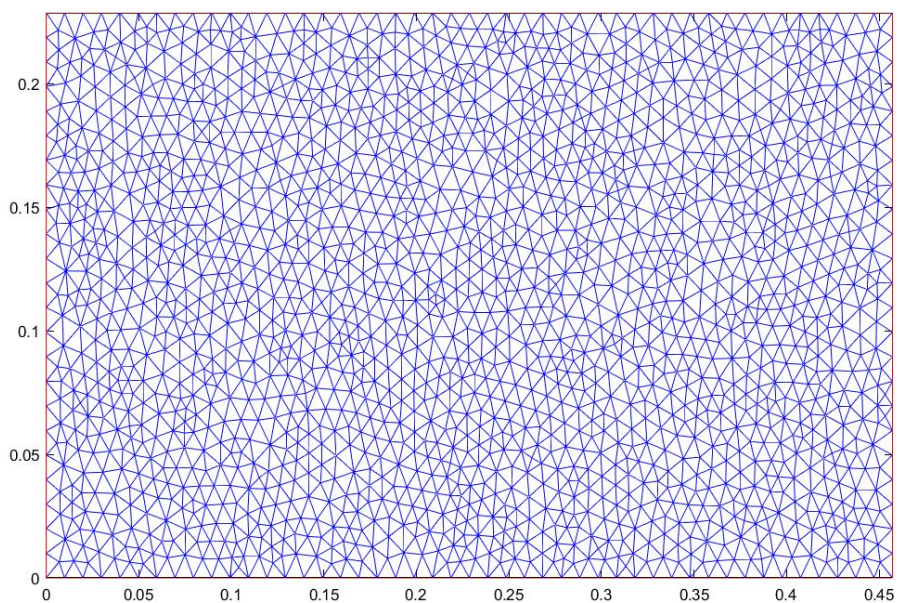


图 4-2 网格 2 划分

网格 3 划分如图 4-3 所示：

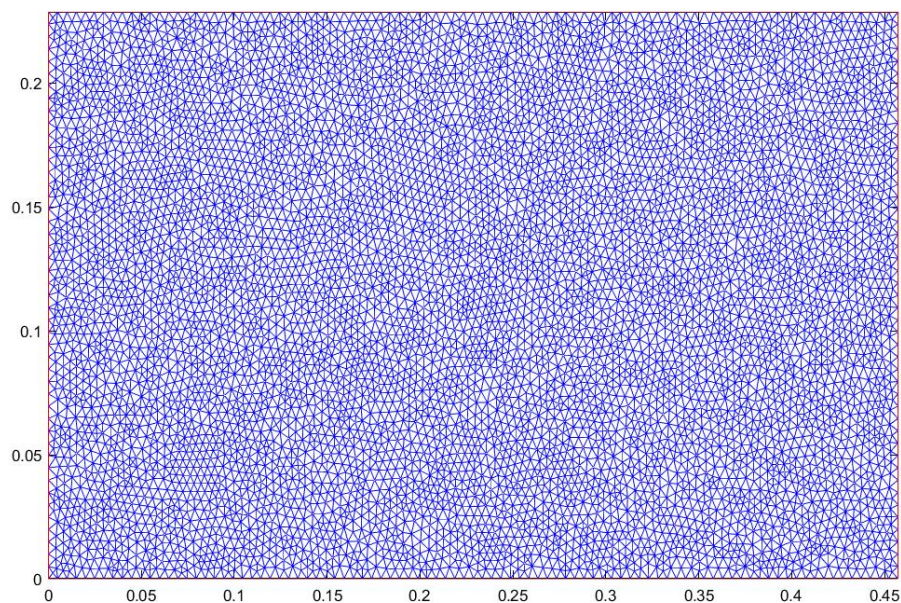


图 4-3 网格 3 划分

可以明显看出，网格划分逐渐变密集，网格 3 最密集。

现分别加载三个网格，利用所写程序得到问题的特征值，并与理论值相对比，计算其相对误差，相对误差计算公式为：

$$E = \frac{|k - k_0|}{k_0} \times 100\%$$

其中 k_0 是理论值。

(2) 理论值计算

理论值结果如表 4-1 所示：

表 4-1 空波导本征值问题理论值

m \ n	0	1	2	3	4
0	0.0000	13.7428	27.4855	41.2283	54.9710
1	6.8714	15.3649	28.3314	41.7969	55.3988
2	13.7428	19.4352	30.7297	43.4584	56.6628
3	20.6141	24.7751	34.3569	46.0946	58.7091
4	27.4855	30.7297	38.8704	49.5502	61.4594

理论值的计算公式如下：

$$k_{cmn} = \sqrt{\left(\frac{m\pi}{a}\right)^2 + \left(\frac{n\pi}{b}\right)^2}$$

理论值表格中的数据较多，手动计算容易出错，可用 **matlab** 根据理论值公式计算出，**matlab** 代码如下：

```
% 计算空波本征值理论值  
a = 0.4572;  
b = 0.2286;  
m = [0,1,2,3,4];  
n = [0,1,2,3,4];  
kc = zeros(5,5);  
  
for i = 1:5  
    for j = 1:5  
        kc(i,j) = pi*((m(i)/a)^2+(n(j)/b)^2)^0.5;  
    end  
end  
disp(kc);
```

运行结果如图 4-4 所示：

```
>> Theoretical_data  
      0    13.7428    27.4855    41.2283    54.9710  
    6.8714    15.3649    28.3314    41.7969    55.3988  
   13.7428    19.4352    30.7297    43.4584    56.6628  
   20.6141    24.7751    34.3569    46.0946    58.7091  
   27.4855    30.7297    38.8704    49.5502    61.4594
```

图 4-4 matlab 程序算出的理论值结果

该数据即为表 4-1 中所示的理论值结果。

(3) 不同剖分下程序计算结果及与理论值的对比

运行程序，得到的 TE、TM 模式前 10 个本征模的传播常数计算值结果如表

4-2、表 4-3 所示：

表 4-2 空波导本征值问题 TE 模式前 10 个本征模的传播常数

序号	模式	理论值	mesh1 实验值	mesh1 相对误差	mesh2 实验值	mesh2 相对误差	mesh3 实验值	mesh3 相对误差
1	TE ₁₀	6.8714	6.8912	0.289%	6.8722	0.012%	6.8716	0.003%
2	TE ₀₁	13.7428	13.8791	0.992%	13.7488	0.044%	13.7443	0.011%
3	TE ₂₀	13.7428	13.9181	1.276%	13.7489	0.045%	13.7443	0.011%
4	TE ₁₁	15.3649	15.5604	1.273%	15.3734	0.056%	15.367	0.014%
5	TE ₂₁	19.4352	19.849	2.129%	19.4523	0.088%	19.4395	0.022%
6	TE ₃₀	20.6141	21.1595	2.646%	20.6351	0.102%	20.6193	0.025%
7	TE ₃₁	24.7751	25.717	3.802%	24.8107	0.144%	24.784	0.036%
8	TE ₀₂	27.4855	28.5828	3.992%	27.5347	0.179%	27.4976	0.044%
9	TE ₄₀	27.4855	28.8503	4.966%	27.5354	0.182%	27.4977	0.044%
10	TE ₁₂	28.3314	29.5823	4.415%	28.386	0.193%	28.3446	0.047%

表 4-3 空波导本征值问题 TM 模式前 10 个本征模的传播常数

序号	模式	理论值	mesh1 实验值	mesh1 相对误差	mesh2 实验值	mesh2 相对误差	mesh3 实验值	mesh3 相对误差
1	TM ₁₁	15.3649	15.5513	1.213%	15.3734	0.056%	15.367	0.014%
2	TM ₂₁	19.4352	19.8501	2.135%	19.4523	0.088%	19.4395	0.022%
3	TM ₃₁	24.7751	25.6831	3.665%	24.8107	0.144%	24.784	0.036%
4	TM ₁₂	28.3314	29.5103	4.161%	28.3833	0.183%	28.3447	0.047%
5	TM ₄₁	30.7297	32.3085	5.138%	30.7972	0.220%	30.7466	0.055%
6	TM ₂₂	30.7297	32.5384	5.886%	30.7975	0.221%	30.7467	0.055%
7	TM ₃₂	34.3569	36.607	6.549%	34.4522	0.277%	34.3805	0.069%
8	TM ₅₁	37.0035	40.2384	8.742%	37.1217	0.319%	37.0328	0.079%
9	TM ₄₂	38.8704	42.2007	8.568%	39.009	0.357%	38.9048	0.089%
10	TM ₁₃	41.7969	45.424	8.678%	41.9631	0.398%	41.8394	0.102%

从表中数据可以看出，即使是 mesh1 这样非常稀疏的划分，求解出的结果与理论值也相当接近，TE 主模相对误差仅有 0.289%，TM 主模相对误差较大一些，但是也只有 1.213%。其他高次模相对误差也在 5%以内。

mesh2 网格的划分较 mesh1 密集了很多，求出的结果也是更加准确，TE 主模相对误差仅有 0.012%，TM 主模相对误差为 0.056%。其他高次模的准确程度也大幅提高，TE 高次模最大相对误差不超过 0.2%，TM 高次模最大相对误差不超过 0.4%，说明程序实现是正确的，有限元方法是合理、准确、有效的。

mesh3 网格的划分最为密集，求出的结果准确程度也远高于 mesh1 和 mesh2，TE 主模相对误差仅有 0.003%，TM 主模相对误差为 0.014%，几乎和理论值一致。其他高次模的解也更加准确，TE 高次模最大相对误差不超过 0.05%，TM 高次模最大误差不超过 0.2%。

根据上述分析，结合表格数据，可以得到如下结论：

①表格横向来看

无论是 TE 模式还是 TM 模式，得到的结果均十分准确，并随着每一次网格划分得更加密集，主模的相对误差均能减小一个数量级，即数值解以一个非常快速的速度收敛于理论值。

这一点可以验证程序的正确性。根据理论分析，划分越密，得到的结果应该越准，越接近于理论值，不存在相对误差的大小震荡的情况，处于一个逐渐收敛的趋势。而表格中的数据刚好体现出了这一点，程序结果准确并且随着划分的密集逐渐收敛于理论值，故程序的编写是正确的。

②表格纵向来看

无论是 TE 模式还是 TM 模式，高次模的准确程度均不如主模，且模次越高，结果准确程度就越低，相对误差越大。

该现象的出现与有限元方法以及高次模的特性有关，有限元方法去解高次模的时候确实存在不够准确的问题。

可以想见，如果网格划分再细致一点，本征值求解结果将会更加精确，与理论值之间的相对误差更小，几乎与理论值相等。

综上所述：用上述程序求解该空波导本征值问题，精度较高，程序正确，结果准确程度相当可观。

2、特征值求解程序效率分析

该程序运算量实际上较大，但是可以通过一些编程上的处理避免一些不必要的计算，减少运算次数，提高运算效率。本程序主要运用了四点处理来提高程序计算的效率：

(1) 尽量减少循环次数

在一个程序中，最费时间的就是循环。求解空波导本征值问题中，需要不断的、反复的遍历所有的三角形、所有的点。本程序在编写过程中，考虑到这一点，故在分析出所有需要循环解决的内容后，尽可能的用一次遍历将他们全部计算出来并存储，以减少循环次数。

再比如在组装矩阵时，一次 i 、 j 循环就将所有用到的基函数以及 A, B 两个矩阵全部算出，避免反复循环计算。

当然，这样的方式属于是用空间换取时间，因为需要将计算出的结果存储起来才能被后面使用。

(2) 矩阵组装时即算即装

矩阵组装函数 `fill_matrix` 采取即算即装模式，在节约空间的同时也节约了时间，提高了效率，关键代码图 4-5 所示：

```
% 矩阵组装
for i = 1:3
    bi = y(L(i,1))-y(L(i,2));
    ci = x(L(i,2))-x(L(i,1));
    for j = 1:3
        bj = y(L(j,1))-y(L(j,2));
        cj = x(L(j,2))-x(L(j,1));

        % 填充A矩阵
        A(t(i,m),t(j,m)) = A(t(i,m),t(j,m)) + (bi*bj+ci*cj)/(4*s);

        % 填充B矩阵
        B(t(i,m),t(j,m)) = B(t(i,m),t(j,m)) + (1+derta(i,j))*s/12;
    end
end
```

图 4-5 矩阵组装函数即算即装代码

可以看出每个小三角形的 A 、 B 矩阵并没有实际的算成 3×3 的矩阵，再进行组装，而是算一个组装一个，因为在组装过程中，仍然需要 i 、 j 的再次循环，不如在算出结果后直接组装到矩阵的对应位置上，这样既不用占用空间去存储 3×3 矩阵，也不需要反复循环，极大地提高了效率。

（3）强加边界条件采用对矩阵降维的方法

添加边界条件原本可以简单的进行：即将 **A**，**B** 矩阵在边界上的点对应位置的行和列均置成 0，然后 **A** 矩阵在该位置再置成 1 即可。这样做的好处是求解出的特征向量是完整的，既有边界上的也有内部的，且边界上的解就是 0，在后续绘图的时候较为方便。

上述方法对矩阵的维度没有影响，效率不会提高。而如果索性将对应位置所在行和列直接删除，对矩阵降维，计算效率的提升也是相当可观的。比如对 mesh3 来说，共有 7347 个点，边界上有 276 个。如果不降维，需要解的是一个 7347×7347 的矩阵方程，而降维后只需要计算 7071×7071 的矩阵方程，维度直接下降 276，效率大幅提升。但是这样也对后续绘图造成了不便，需要将删除的为 0 的值补充回来才能正确绘图。

强加边界条件对矩阵降维的关键代码如图 4-6 所示：

```
Nnode = size(A);  
  
inner_node_num = 1:Nnode;  
inner_node_num(e(1,:)) = [];  
  
A = A(inner_node_num, inner_node_num);  
B = B(inner_node_num, inner_node_num);
```

图 4-6 强加边界条件对矩阵降维的关键代码

（4）求解广义特征值采用稀疏矩阵

在求解广义特征方程时，基本思想是用 eig 函数去求解，但是实测 eig 函数求解较慢，这与 matlab 对 eig 函数求解方式的设定有关。为减少运算时间，先将满秩矩阵 **A** 与 **B** 转化为稀疏矩阵，再用 eigs 函数求解，可以极大提高效率。

采用稀疏矩阵求解关键代码如图 4-7 所示：

```
A = sparse(A);  
B = sparse(B);  
[Ez, D] = eigs(A, B, num+1, 'smallestabs');
```

图 4-7 采用稀疏矩阵求解关键代码

(5) 程序实际运行时间与效率分析

程序利用 mesh1 的划分得到解的运行时间如图 4-8 所示：

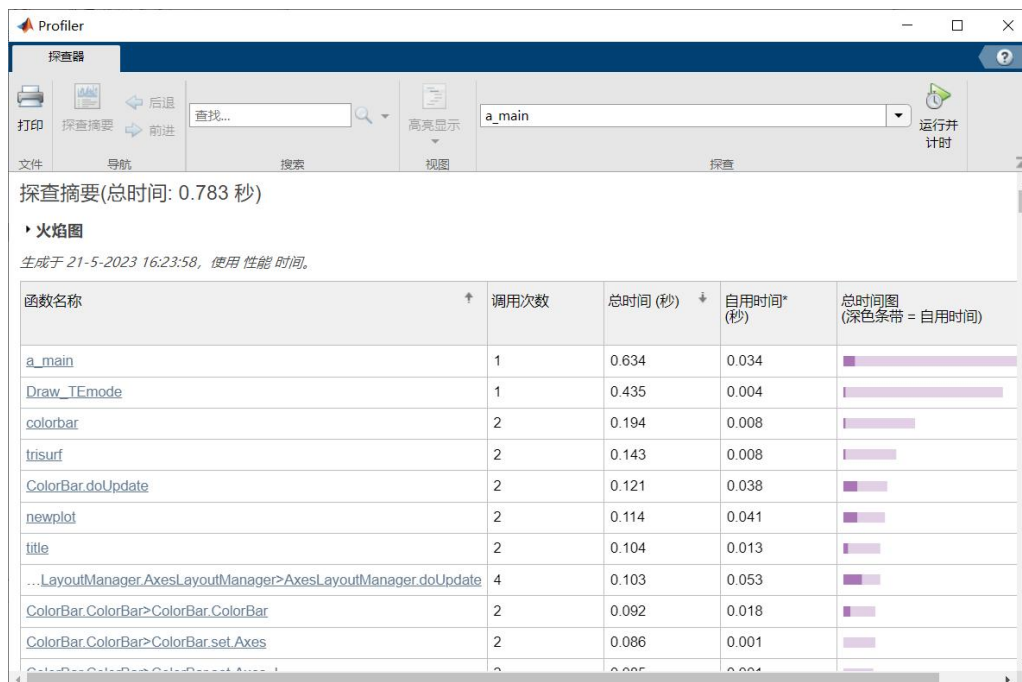


图 4-8 mesh1 程序运行时间

由图可知，Draw_TEmode 函数占据了大部分的运行时间。如果仅需要得到本征值，程序运行时间如图 4-9 所示：

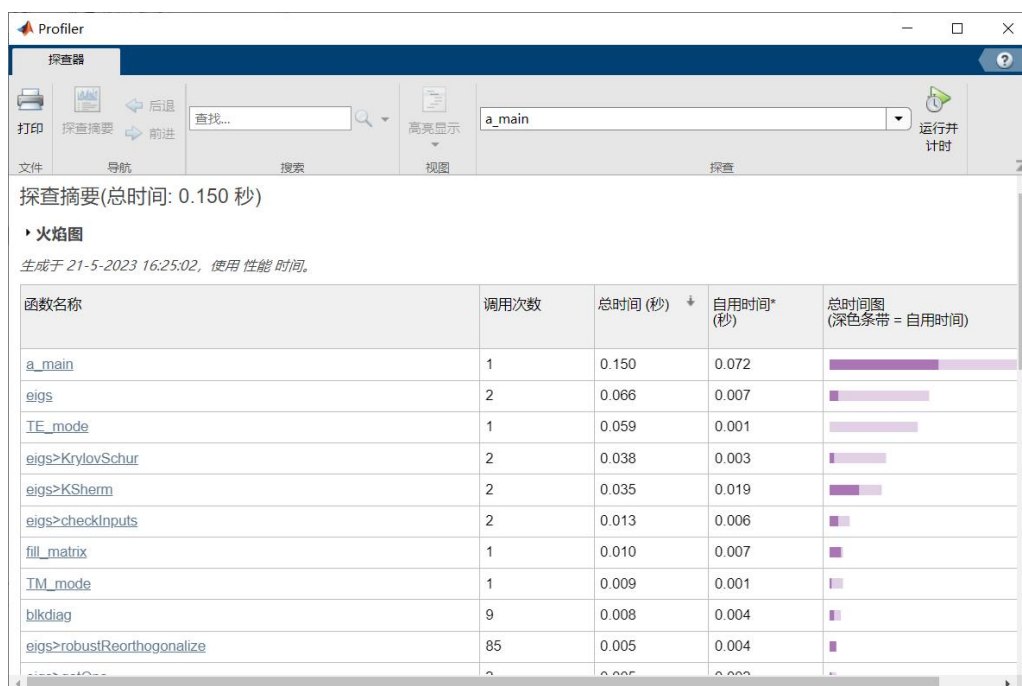


图 4-9 mesh1 程序运行时间 (仅求出本征值)

程序利用 mesh2 的划分得到解的运行时间如图 4-10 所示：

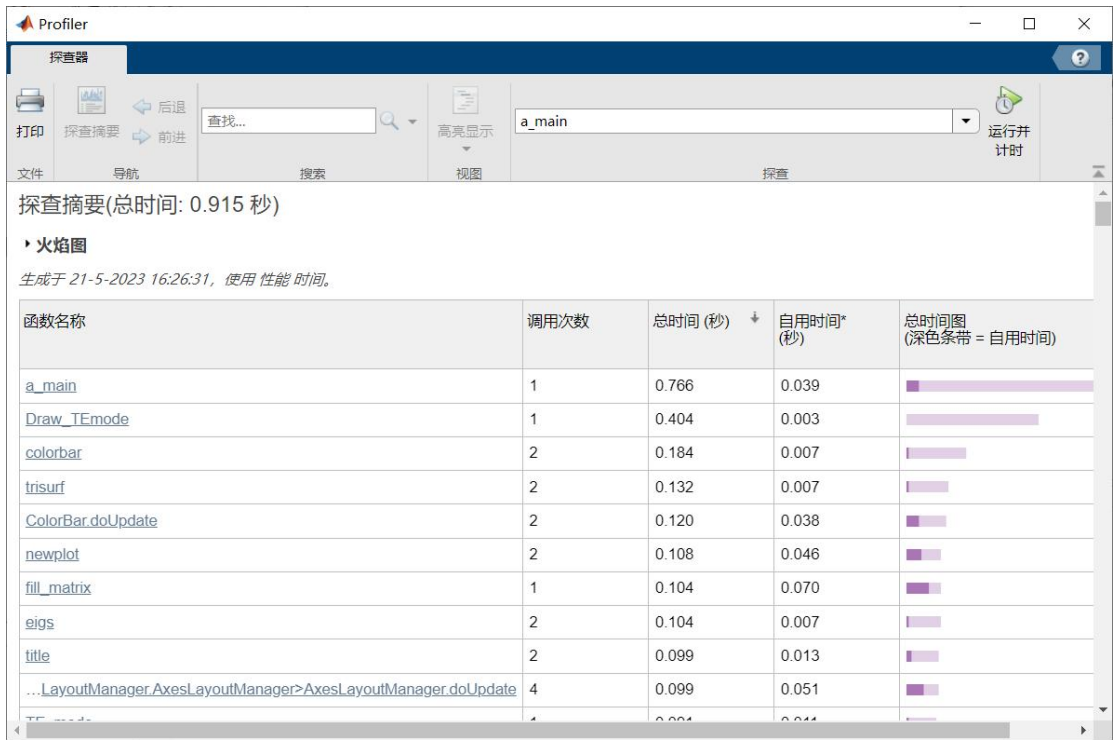


图 4-10 mesh2 程序运行时间

如果仅需要得到本征值，程序运行时间如图 4-11 所示：

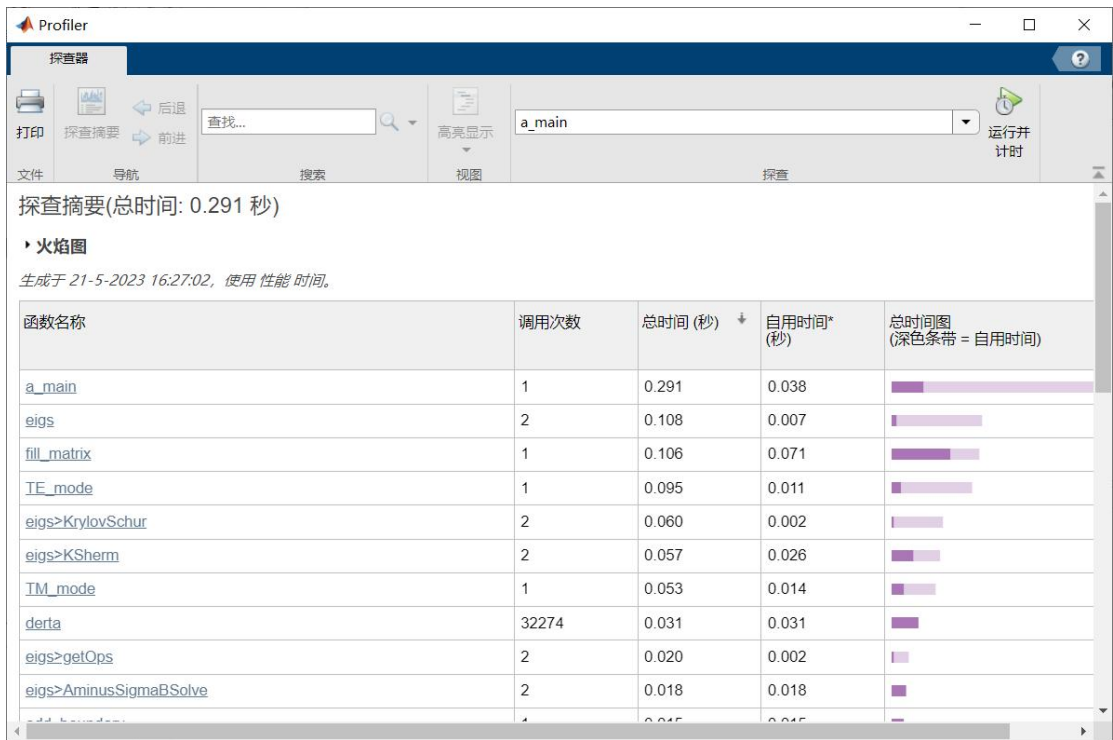


图 4-11 mesh2 程序运行时间 (仅求出本征值)

程序利用 mesh3 的划分得到解的运行时间如图 4-12 所示：

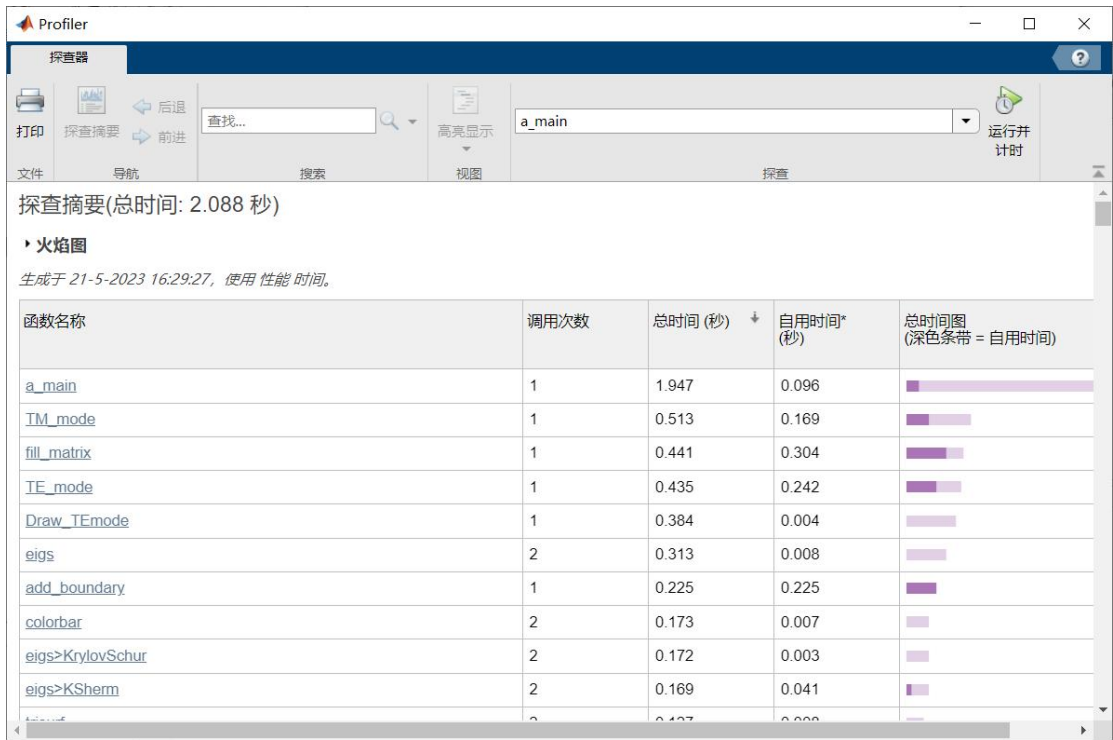


图 4-12 mesh3 程序运行时间

如果仅需要得到本征值，程序运行时间如图 4-13 所示：

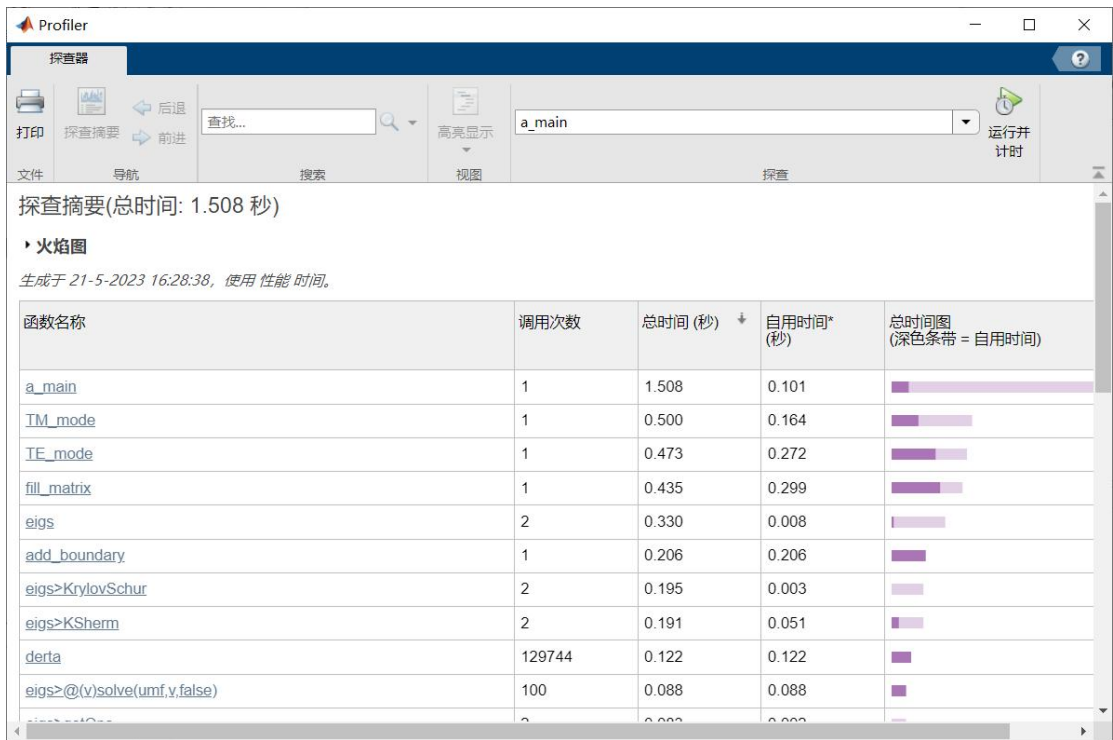


图 4-13 mesh3 程序运行时间 (仅求出本征值)

由图可知，无论是利用哪个网格，在仅需求出本征值的条件下，矩阵填充函数 `fill_matrix` 函数占据了大部分时间，这是符合直觉与实际的，因为在代码分析的时候讨论过：`fill_matrix` 函数计算量较大。

从图中还可以看出，解广义特征方程的部分用时并不是很多，最长不过 **0.2s** 这与已经将其转换成稀疏矩阵再用 `eigs` 函数计算有直接关系，极大提高了效率。

将程序运行时间整理为表格，如表 4-4 所示：

表 4-4 矩形空波导本征值问题有限元求解程序运行时间

网格	程序运行时间（s）	
	绘出场分布 （同时求出本征值）	不绘出场分布 （仅得到本征值）
mesh1	0.783	0.150
mesh2	0.915	0.291
mesh3	2.088	1.508

对表格数据进行横向对比，可以发现绘出场分布还是很花时间的一件事，但是对我们直观的观察电场的传播模式，以及其大小的分布，更好的理解矩形空波导本征值问题具有重要意义。虽然很费时间，但是也几乎都在 **2s** 以内，一瞬间的事情。如果仅需要得到本征值，除 `mesh3` 外程序运行时间可以降低到 **0.3s** 左右，几乎运行后瞬间就能出结果，效率极高。

对表格数据进行纵向对比，可以发现当网格划分变密后，程序运行时间均发生了较为显著的增加，这也是符合物理直觉的，因为划分更密了，运算的数据更多，运行时间自然变长。但即使如此，运行时间基本都在 **2s** 以内，时间尺度很小，几乎感受不出来。

如果将网格进行更加细致的划分，程序运行时间将会继续增加。但是通过前面的精度分析可知，`mesh3` 的准确程度已经很高了，相对误差仅有 **0.1%** 以内，如果将划分变得更密，对精度的影响几乎感受不出来。但是通过由 `mesh2` 到 `mesh3` 时间增大的幅度可知，网格变密将会使运行时间大幅增加，此时精度不会增大太多，运行时间变长，效率下降，得不偿失，`mesh3` 的划分已经足够。

综上所述：通过一些编程上的处理，使得该程序具有极高的效率，能够快速解决矩形空波导本征值问题。

3、所解得的各模式下电场分布图

该程序在求解时，可以通过参数 `mode_All` 来设置求解模式的个数，这里按照课程要求求解前 10 个。

由于 `mesh3` 划分最密，结果最准，所以以下各个模式的电场分布图均使用 `mesh3` 的划分计算出的结果。

(1) TE 模

TE 模的前十个场分布如图 4-14——4-24 所示：

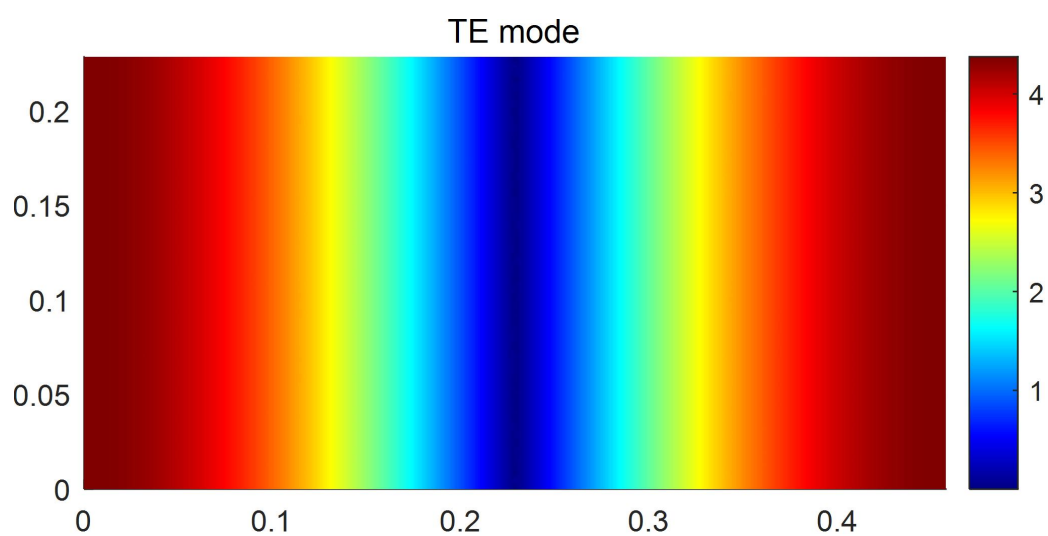


图 4-14 1_TE₁₀ 模 (TE 主模)

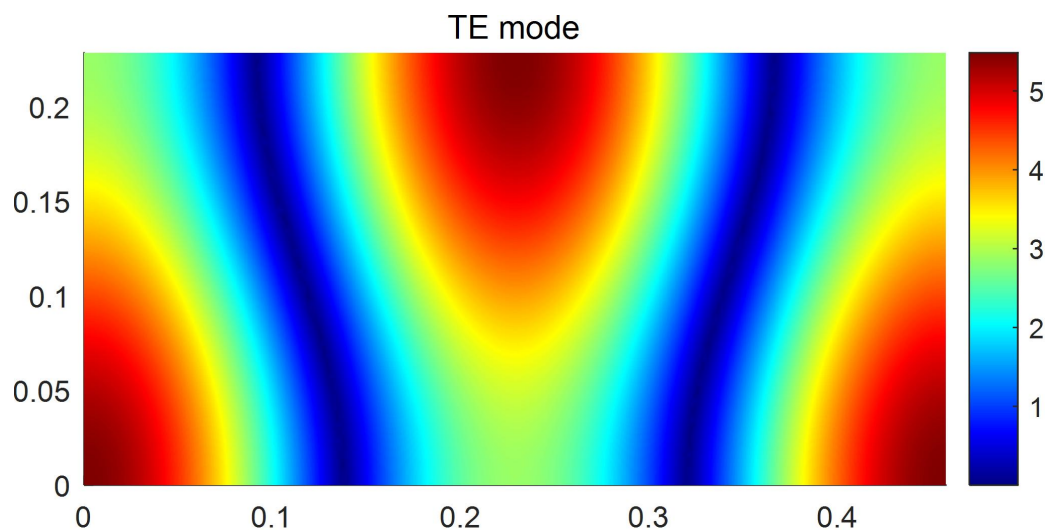


图 4-15 2_TE₀₁ 模

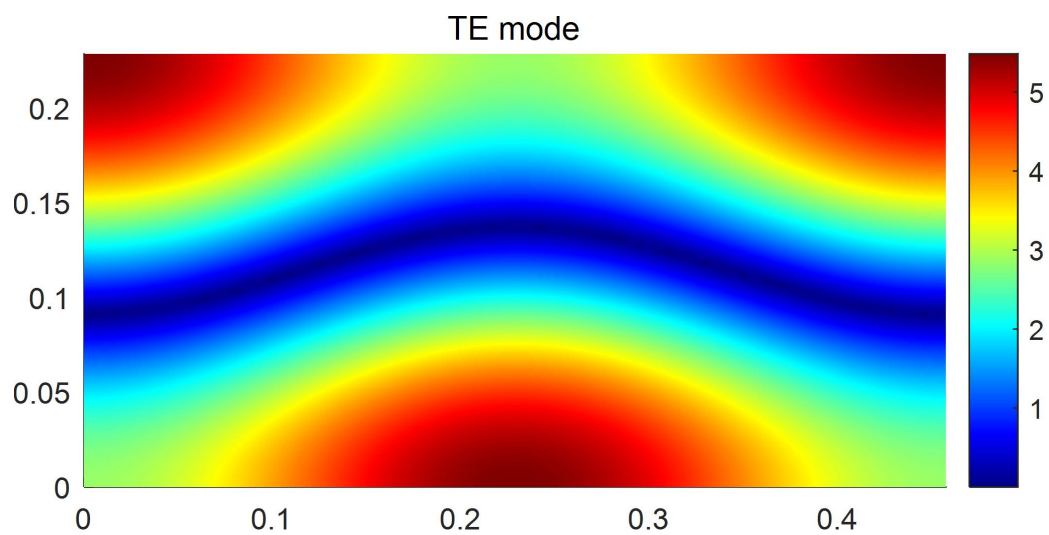


图 4-16 3-TE₂₀ 模

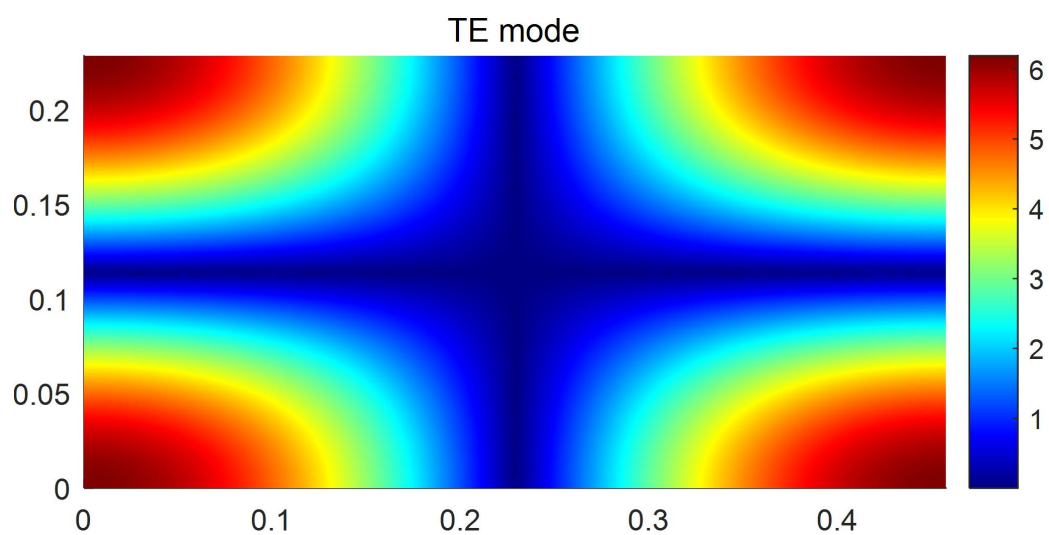


图 4-17 4-TE₁₁ 模

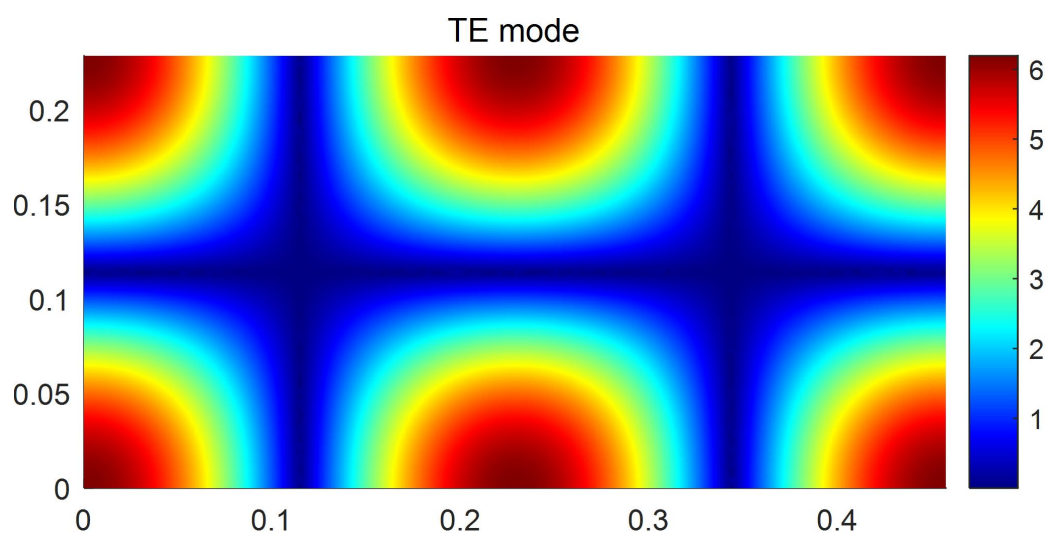


图 4-18 5-TE₂₁ 模

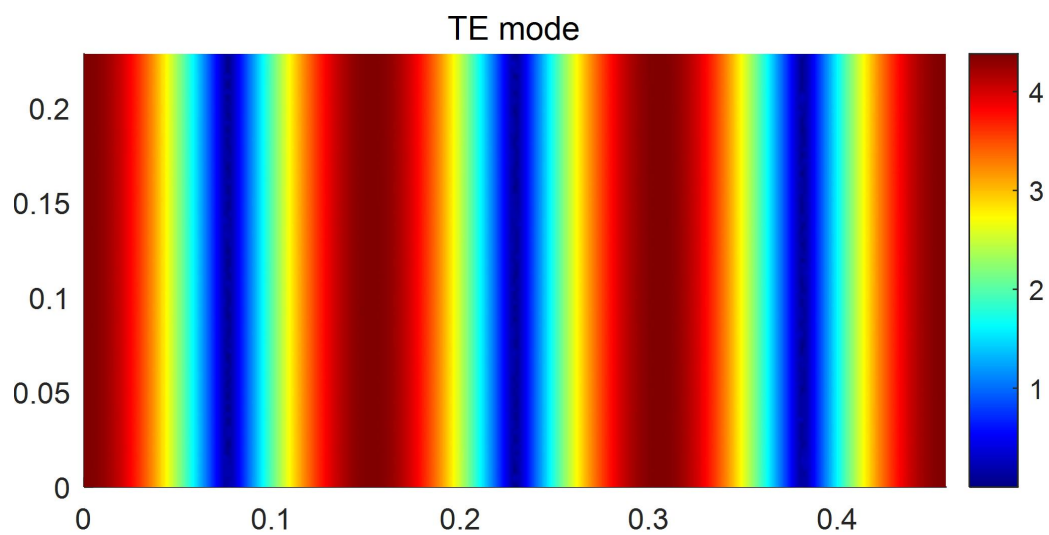


图 4-19 6-TE₃₀ 模

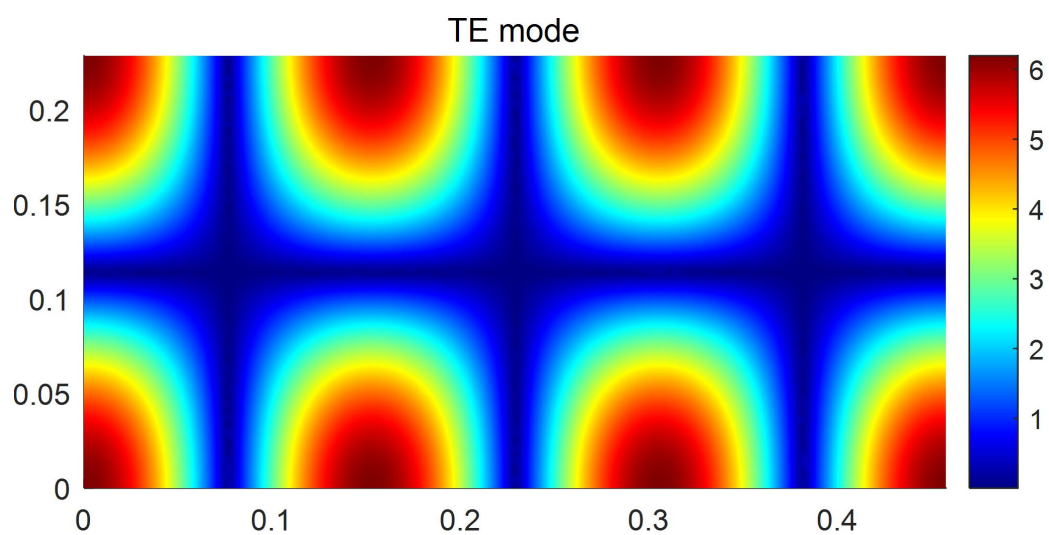


图 4-20 7-TE₃₁ 模

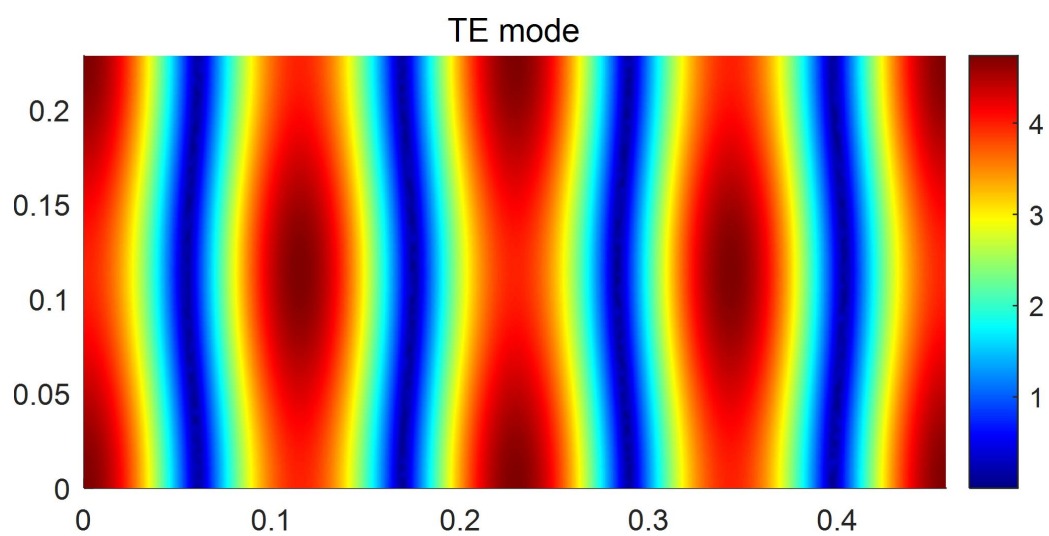


图 4-21 8-TE₀₂ 模

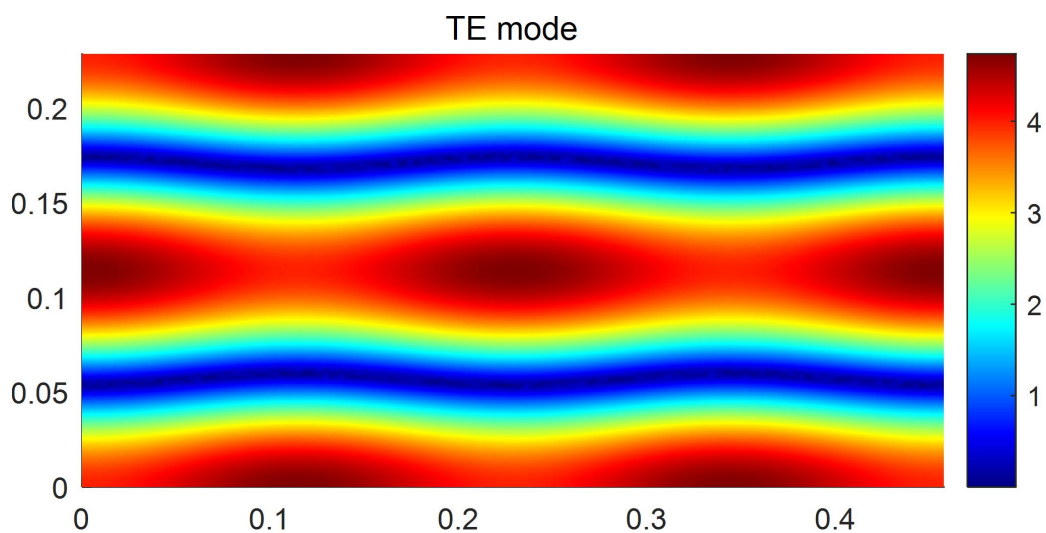


图 4-22 9-TE₄₀ 模

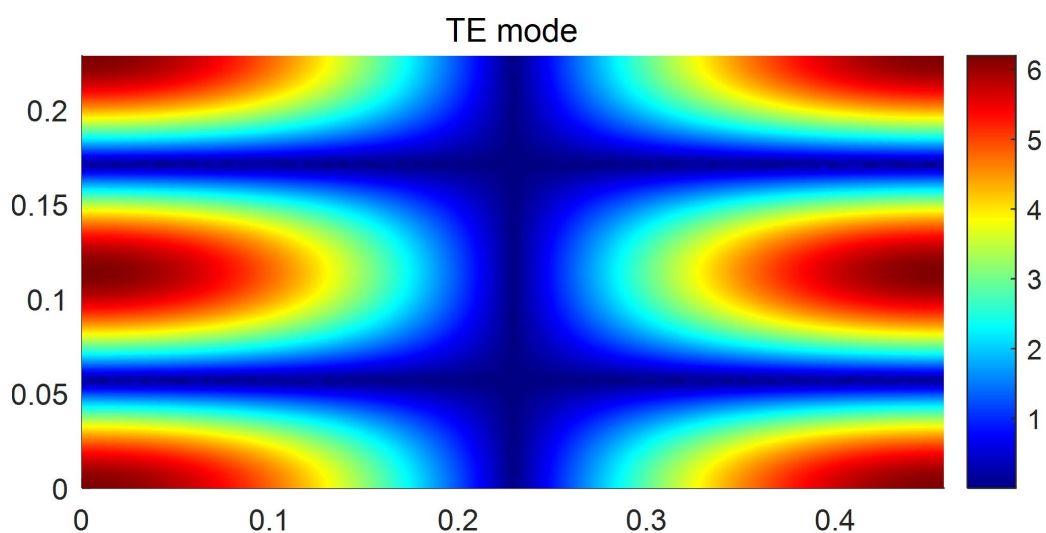


图 4-23 10-TE₁₂ 模

图中的不同颜色代表 z 方向场的大小，颜色越偏近于蓝色电场越小，越偏近于红色电场越大。

上述 10 个 TE 模式的场分布均与理论解图像相一致。

(2) TM 模

TM 模的前十个场分布如图 4-24——4-33 所示：

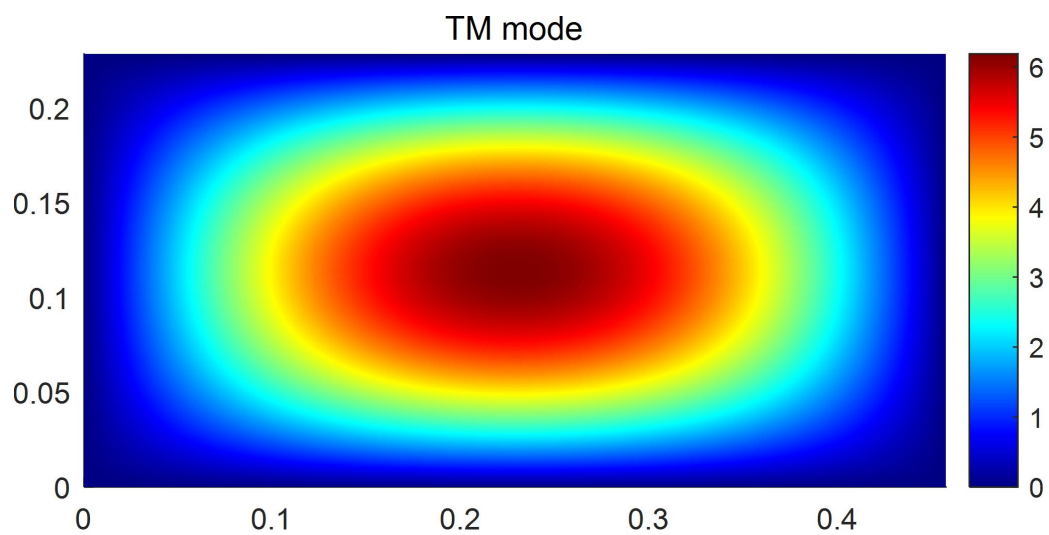


图 4-24 $1_{\text{TM}_{11}}$ 模 (主模)

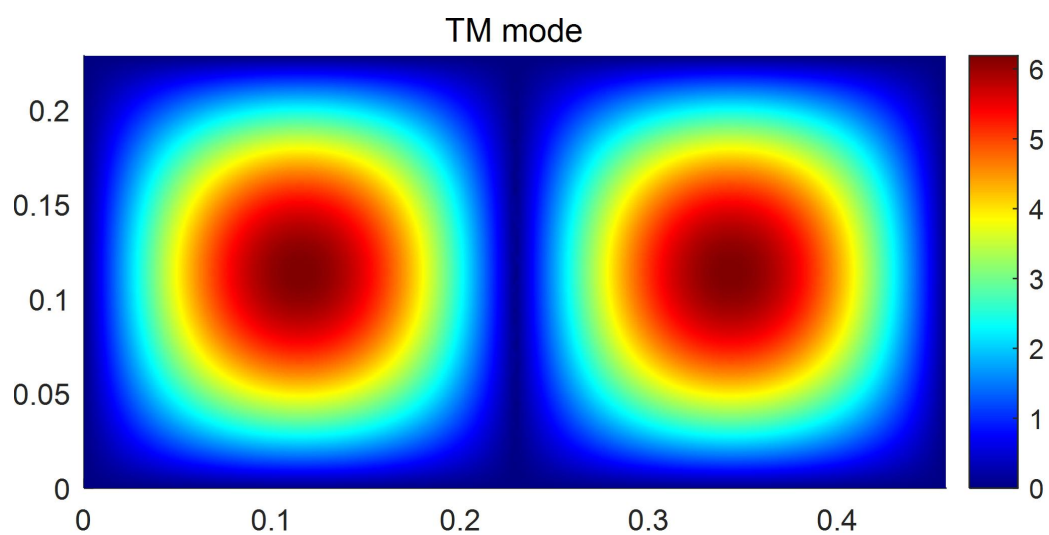


图 4-25 $2_{\text{TM}_{21}}$ 模

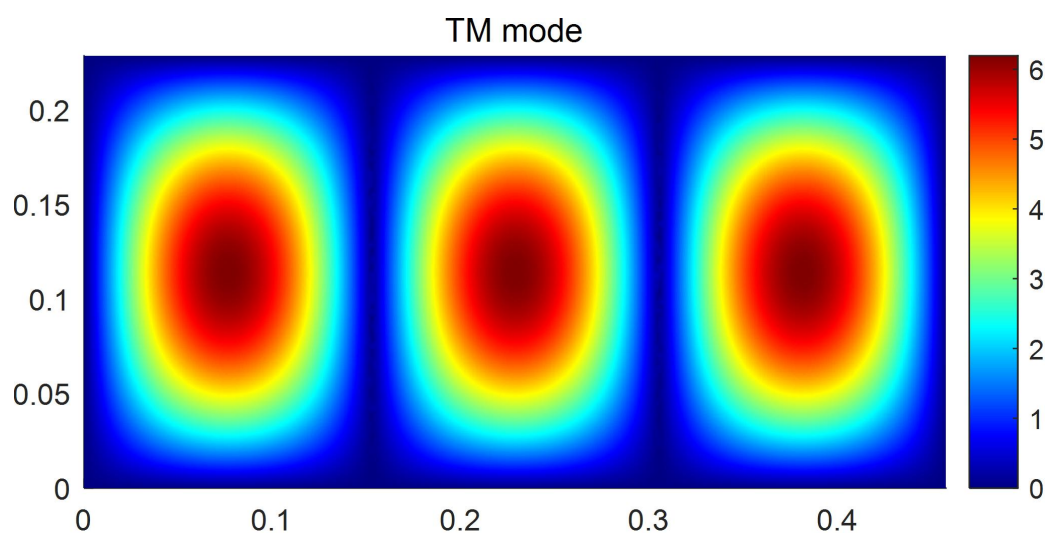


图 4-26 $3_{\text{TM}_{31}}$ 模

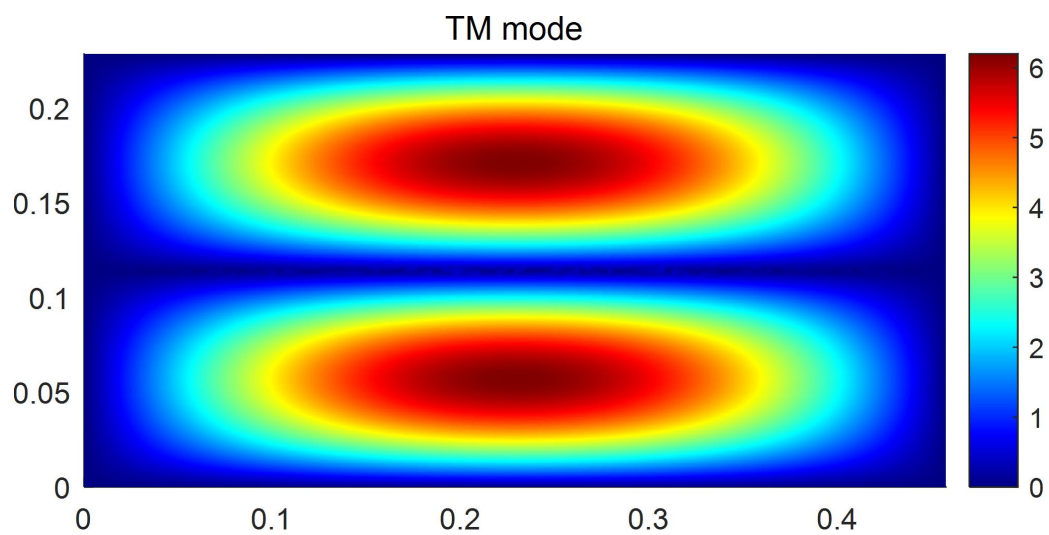


图 4-27 4_TM_{12} 模

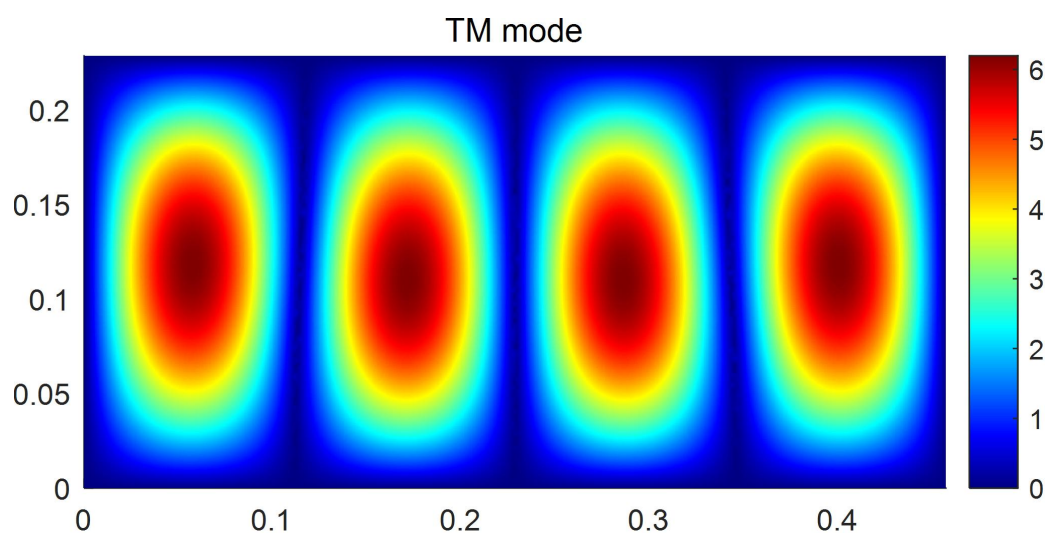


图 4-28 5_TM_{41} 模

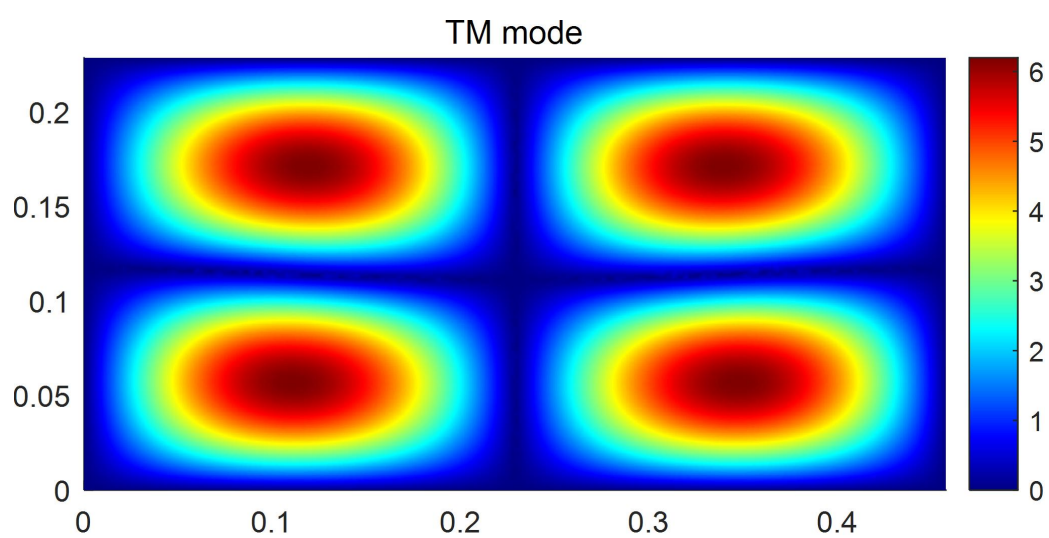


图 4-29 6_TM_{22} 模

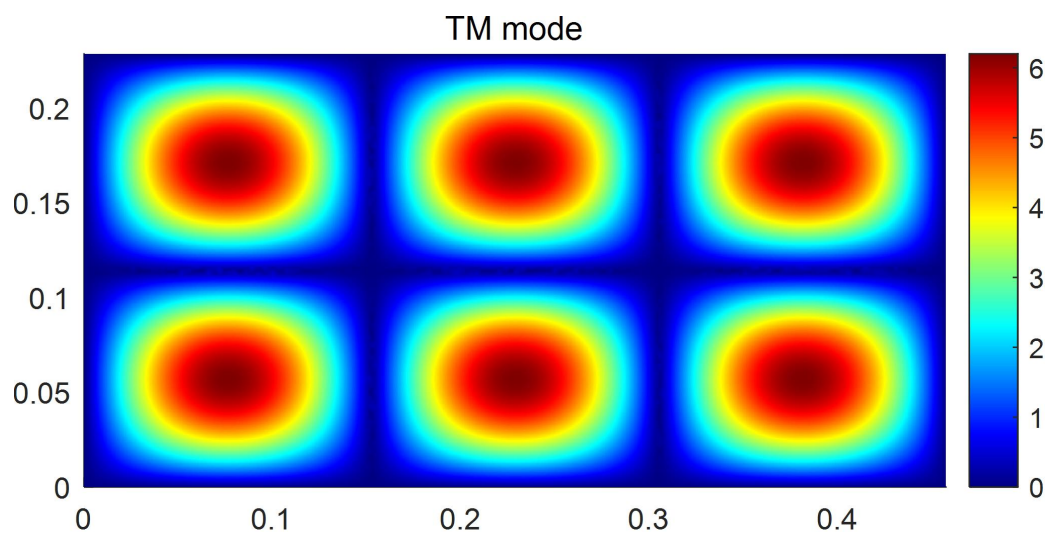


图 4-30 7_{TM32} 模

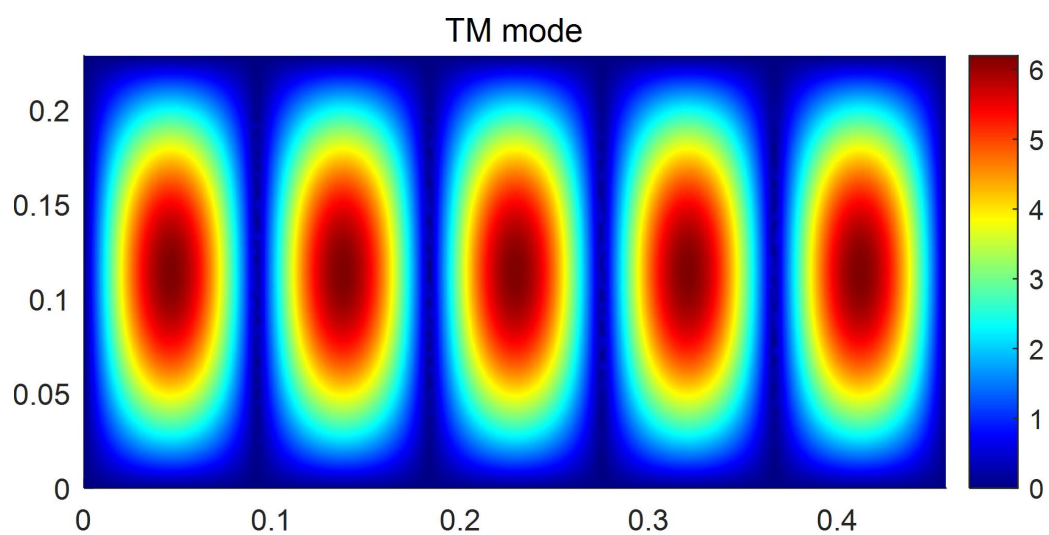


图 4-31 8_{TM51} 模

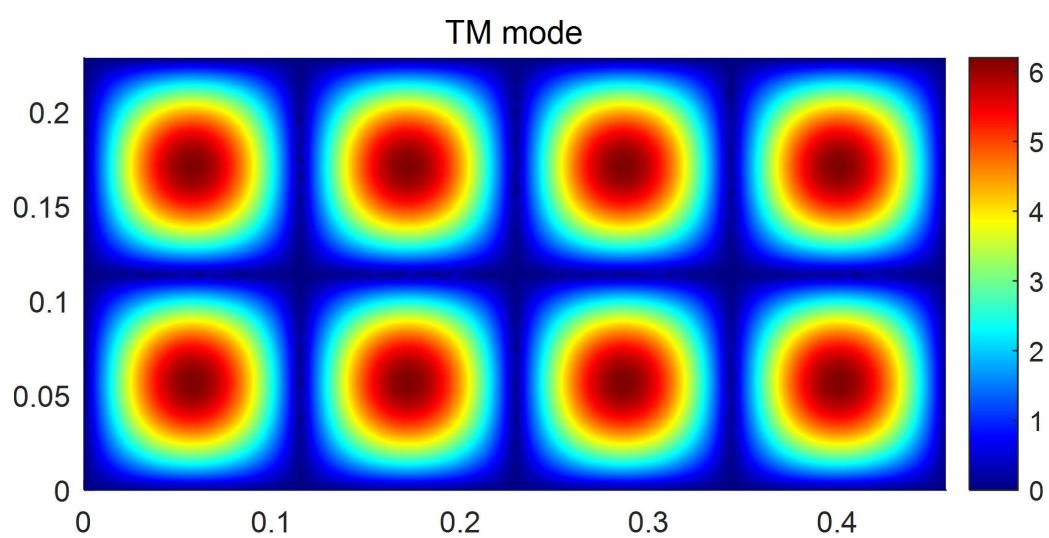


图 4-32 9_{TM42} 模

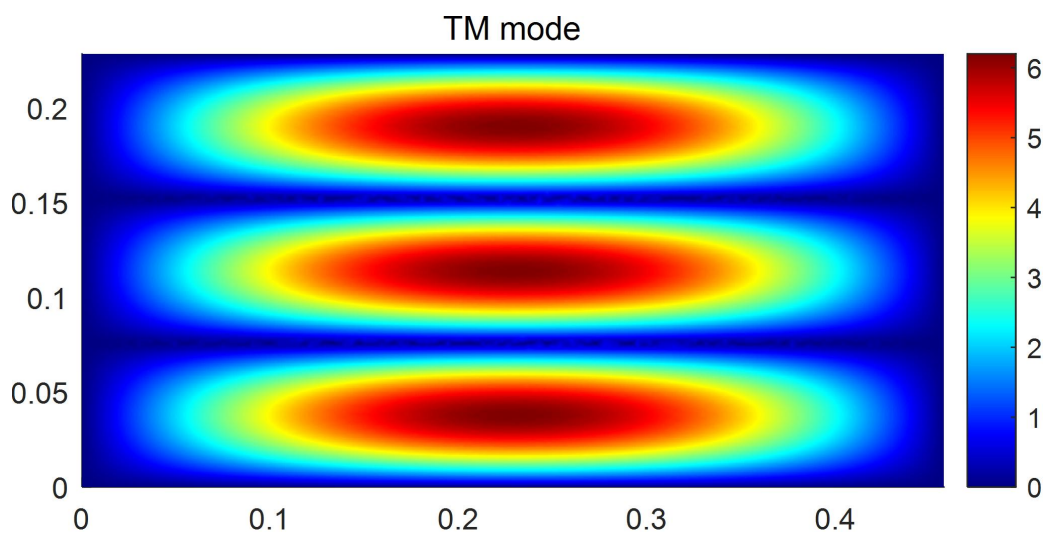


图 4-33 10_TM_{10} 模

图中的不同颜色代表 z 方向场的大小，颜色越偏近于蓝色电场越小，越偏近于红色电场越大。

上述 10 个 TM 模式的场分布均与理论解图像相一致。

五、利用上述程序求解不规则波导的传播常数

通过上述对存在理论解的矩形空波导的本征值求解，以及与理论值之间的对比，已经证明该程序编写正确，能够准确、高效的计算空波导的本征值问题。下面将用上述程序求解一不规则波导的前 5 个本征模的传播常数。

该不规则波导的形状及参数如图 5-1 所示：

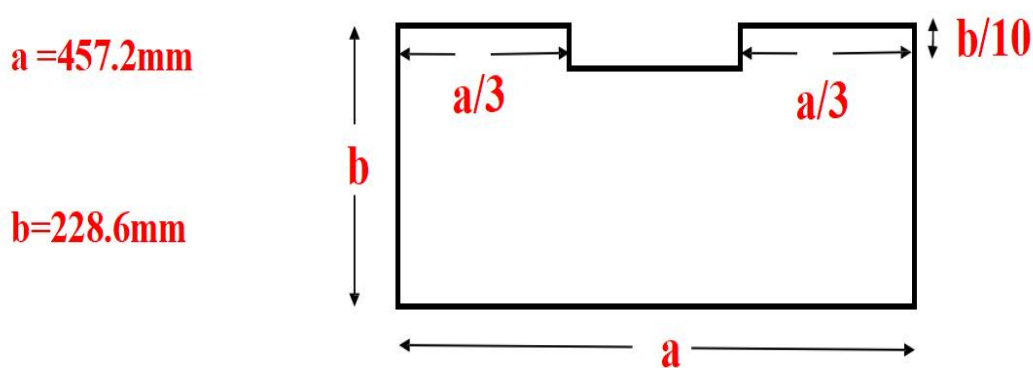


图 5-1 不规则波导的形状及参数

1、特征值求解

(1) 网格划分

这里采用文件夹中 `waveguide_2_mesh.m` 文件生成的网格，该网格如图 5-2 所示：

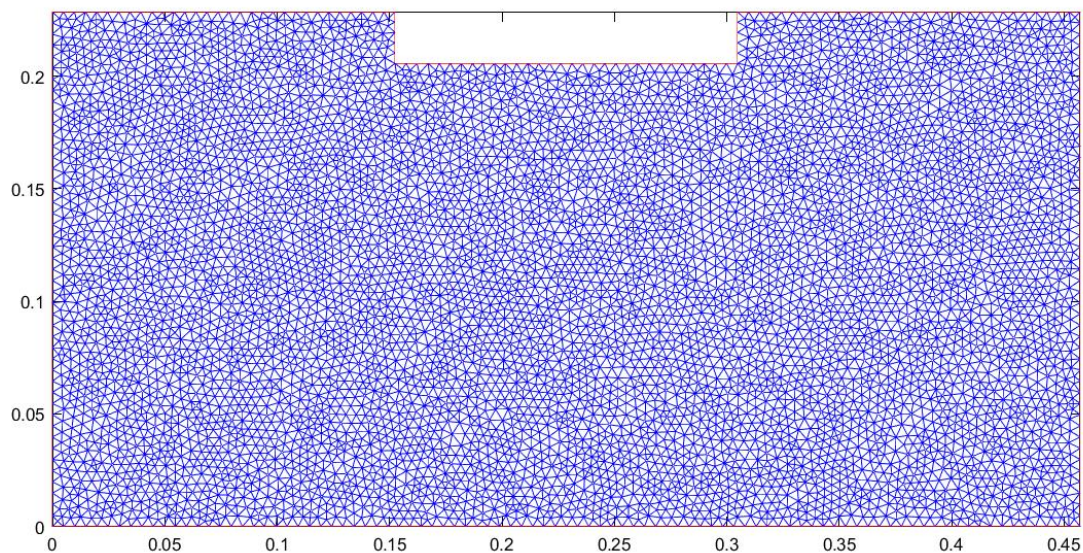


图 5-2 `waveguide_2_mesh.m` 文件生成的网格

由图可以看出该网格划分较密，这是为了在求解过程中得到更加准确的本征值结果与图像。

(2) 求解结果

在主程序中注释掉其他三个网格，运行第四个网格，如图 5-3 所示：

```
% 加载网格
% load('waveguide_1_mesh1');
% load('waveguide_1_mesh2');
% load('waveguide_1_mesh3');
load('waveguide_2_mesh');
```

图 5-3 加载并运行第四个网格

得到的该不规则波导前 5 个本征值如表 5-1 所示：

表 5-1 不规则波导前 5 个本征模的传播常数

模式	主模	第一高次模	第二高次模	第三高次模	第四高次模
TE	6.6126	13.5682	14.4212	15.2214	20.1672
TM	16.3661	19.8157	25.0459	30.3323	30.7708

可以看出，该结果似乎与之前求解的矩形波导的前 5 个本征值较为接近。下面将对对比该不规则波导与矩阵波导的前 5 个本征值，如表 5-2 所示：

表 5-2 不规则波导与矩形空波导前 5 个本征模的传播常数对比

模式	主模	第一高次模	第二高次模	第三高次模	第四高次模
TE	6.6126	13.5682	14.4212	15.2214	20.1672
TE（矩形）	6.8714	13.7428	13.7428	15.3649	19.4352
TM	16.3661	19.8157	25.0459	30.3323	30.7708
TM（矩形）	15.3649	19.4352	24.7751	28.3314	30.7297

由表 5-2 中数据可以看出，该不规则波导的前 5 个本征模的传播常数确实与之前求解的矩形波导的前五个本征值较为接近，差距基本都在 1 以内，最大差距不超过 2。

事实上，这一点的出现是必然的。因为该不规则波导，实际上就是将之前的矩形波导抠下去了一块，而且抠下去的部分特别小，对于波导整体形状的影响并不是很大。所以，该不规则波导的前 5 个本征模的传播常数与之前求解的矩形波导的前五个本征值较为接近。

2、程序运行时间与效率分析

求解该不规则波导的前 5 个本征模的传播常数程序运行时间如图 5-4 所示：

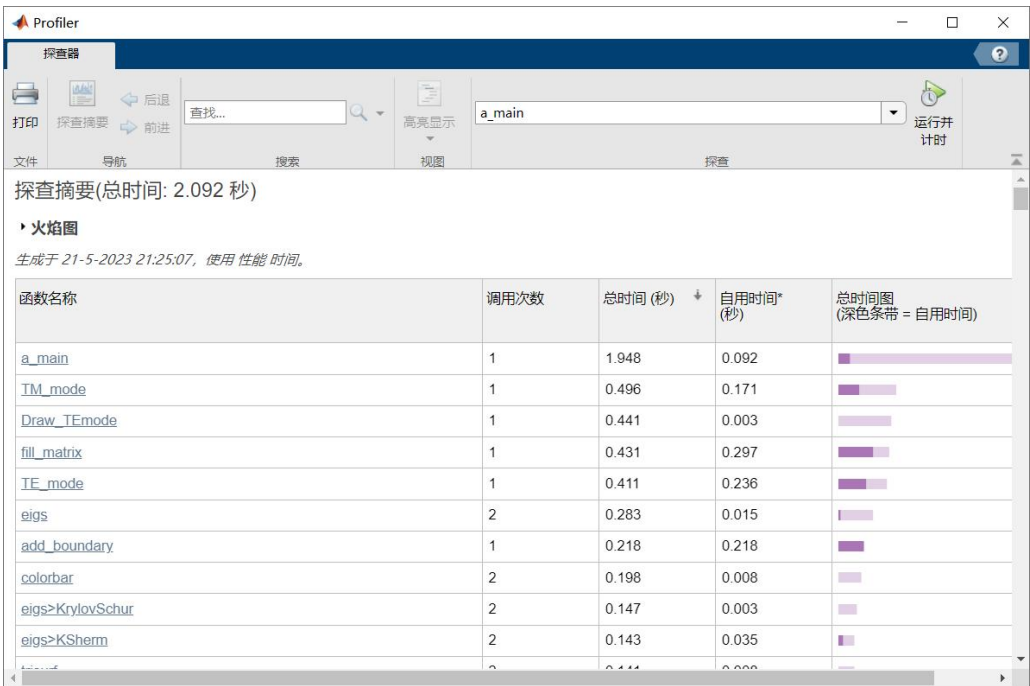


图 5-4 求解该不规则波导的前 5 个本征模的传播常数程序运行时间

如果仅需要得到本征值，程序运行时间如图 5-5 所示：

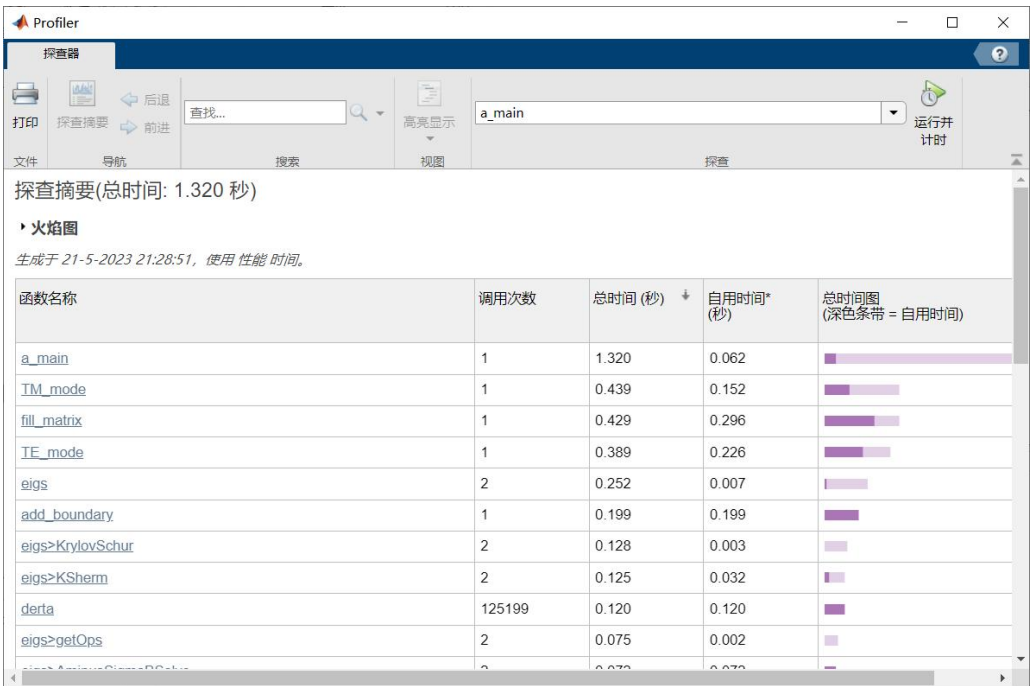


图 5-5 求解该不规则波导的前 5 个本征模的传播常数程序运行时间（仅求出本征值）

将上述结果整理成表格，如表 5-3 所示：

表 5-3 求解不规则波导前 5 个本征模的传播常数程序运行时间

网格	程序运行时间（s）	
	绘出场分布 （同时求出本征值）	不绘出场分布 （仅得到本征值）
不规则波导	2.092	1.320

由图 5-2 可以看出，对该波导的划分是非常稠密的。通过 matlab 中的数据可知，共有 7100 个点参与该不规则波导的划分，划分出的三角形更是达到了 13911 个，如图 5-6 所示：



图 5-6 划分该网格的点数与划分出的小三角形数

这是一个十分庞大的数字，计算机在求解过程中运算量巨大，但即使这么大的计算量，程序依然能够在 2s 左右这样一个对我们而言一瞬间的时间跑完，足以说明该程序具有极高的效率。

3、绘制该不规则波导的场分布图像

在绘制之前，可以做如下猜想：

该不规则波导是将之前的矩形波导抠下去了一块，而且抠下去的部分特别小，对于波导整体形状的影响并不是很大。在特征值求解部分，已经做过对比，该不规则波导的前 5 个本征模的传播常数与之前求解的矩形波导的前五个本征值较为接近。

既然如此，可以猜想该不规则波导的前 5 个本征模的场分布也与矩形波导的前 5 个本征模的场分布类似。

(1) TE 模

该不规则波导前 5 个 TE 模的场分布如图 5-7——5-11 所示：

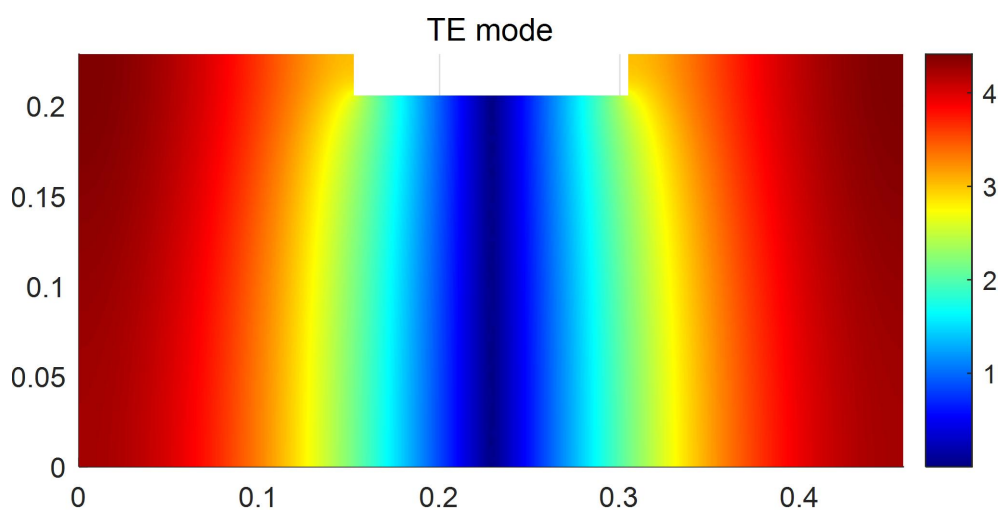


图 5-7 TE 主模

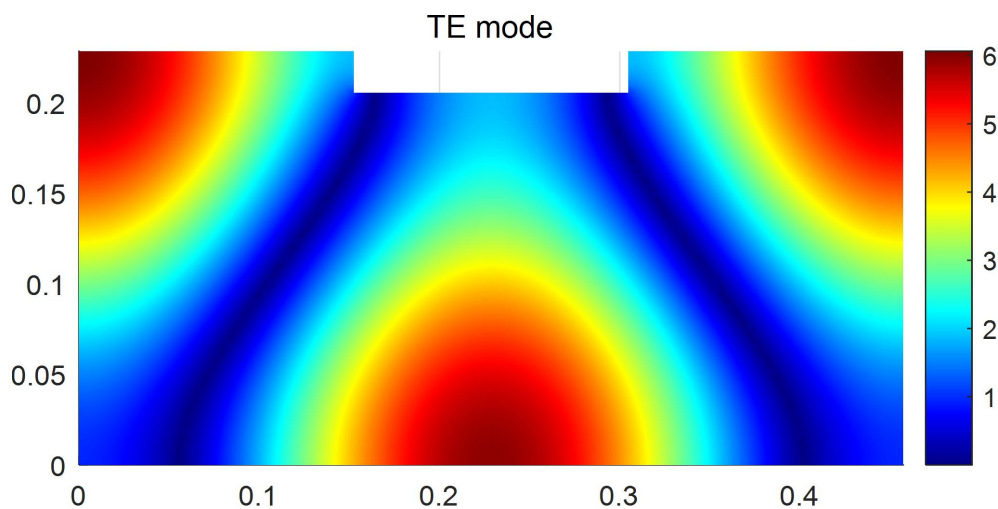


图 5-8 TE 第一高次模

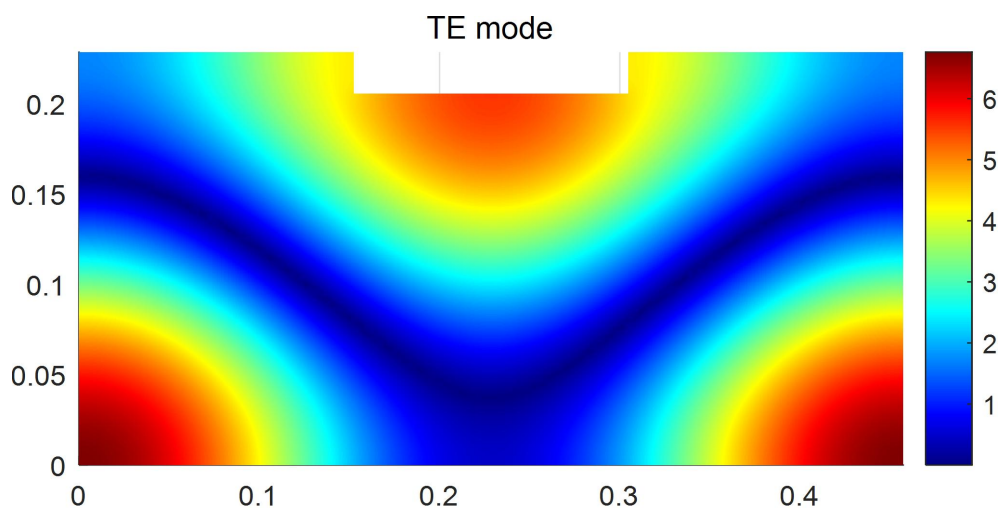


图 5-9 TE 第二高次模

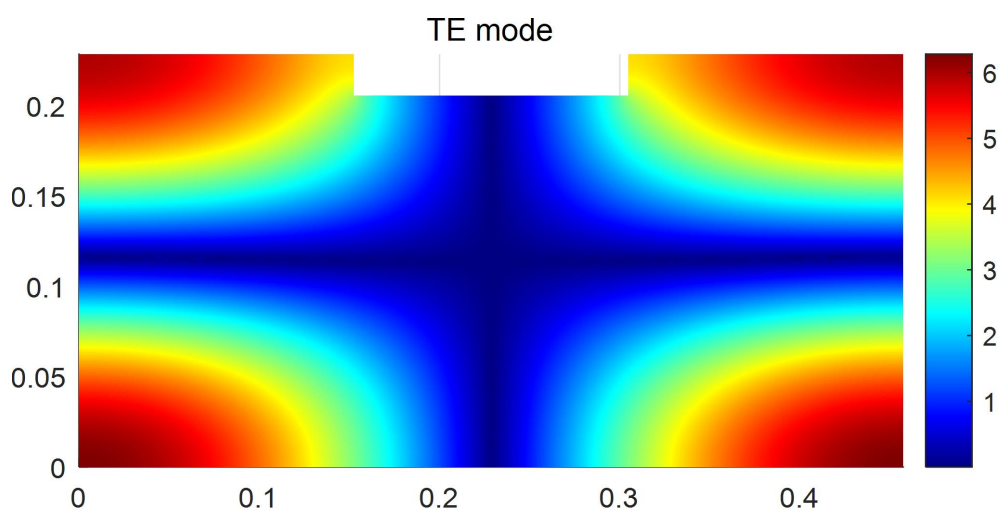


图 5-10 TE 第三高次模

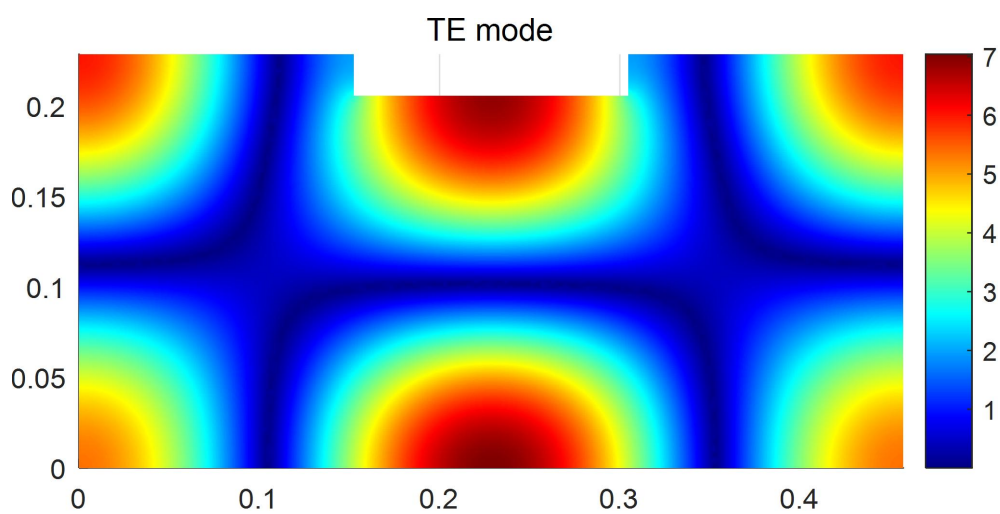
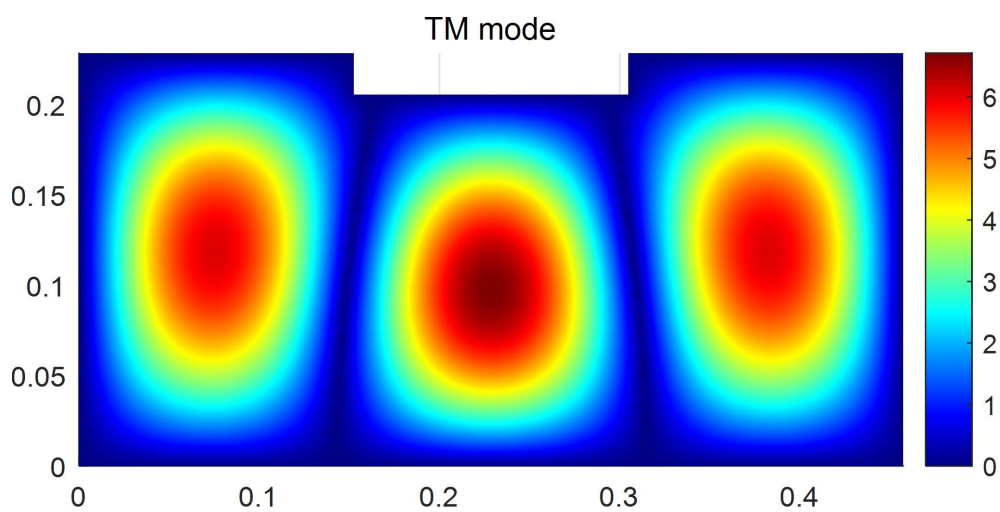
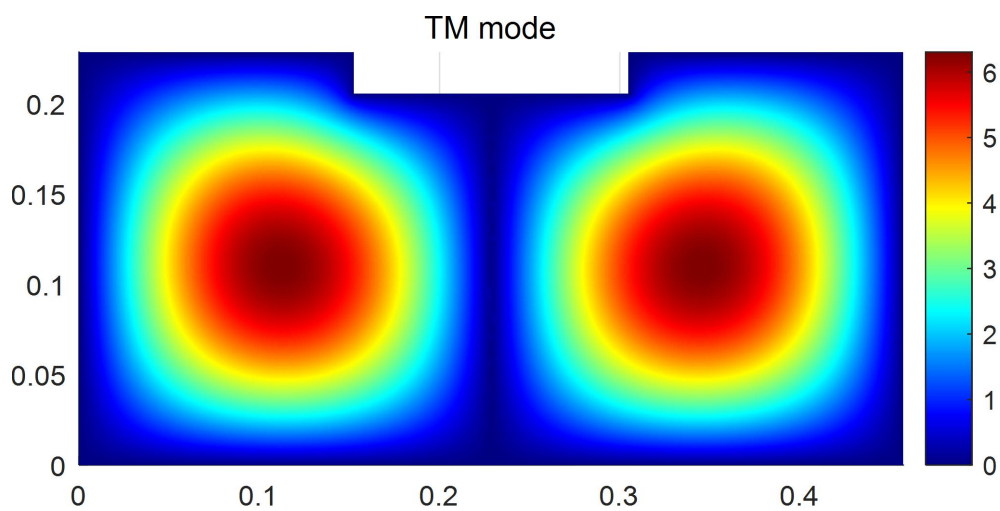
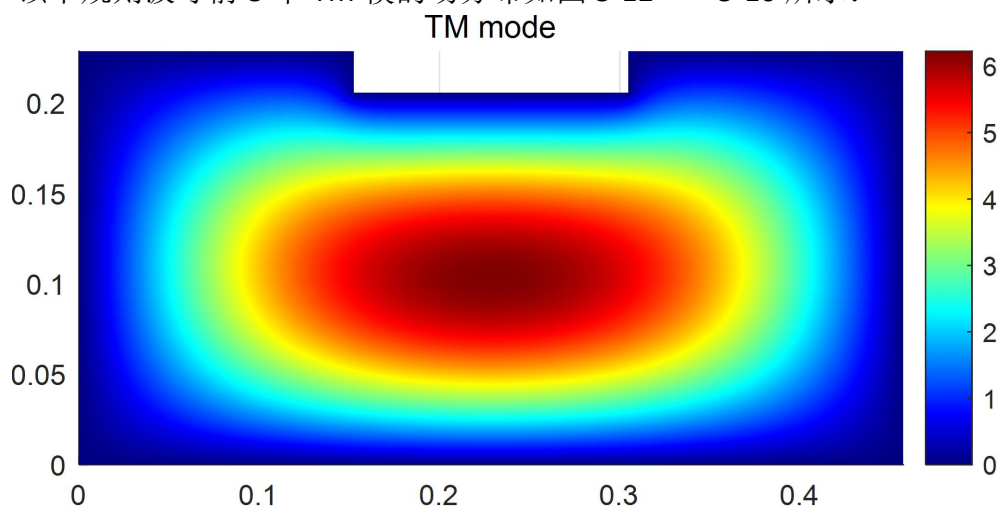


图 5-11 TE 第四高次模

(2) TM 模

该不规则波导前 5 个 TM 模的场分布如图 5-12——5-16 所示：



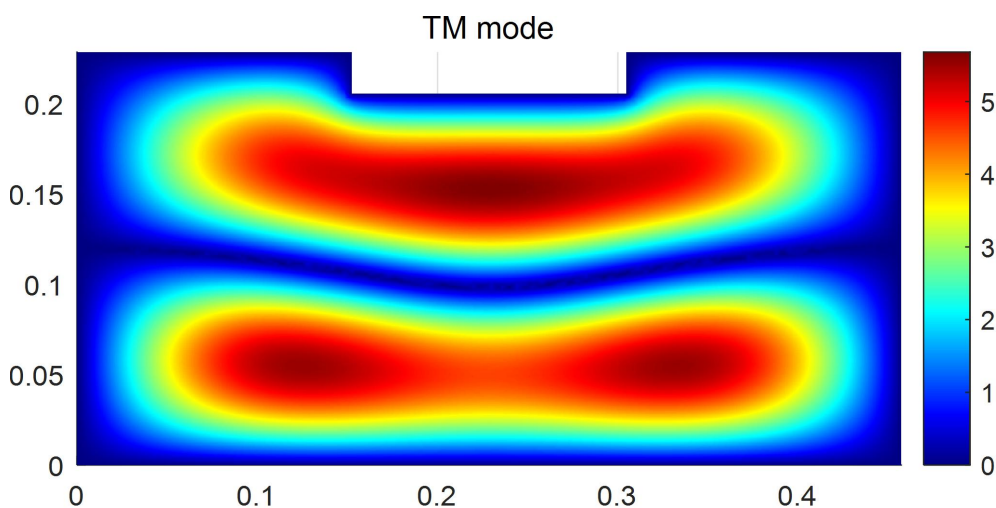


图 5-15 TM 第三高次模

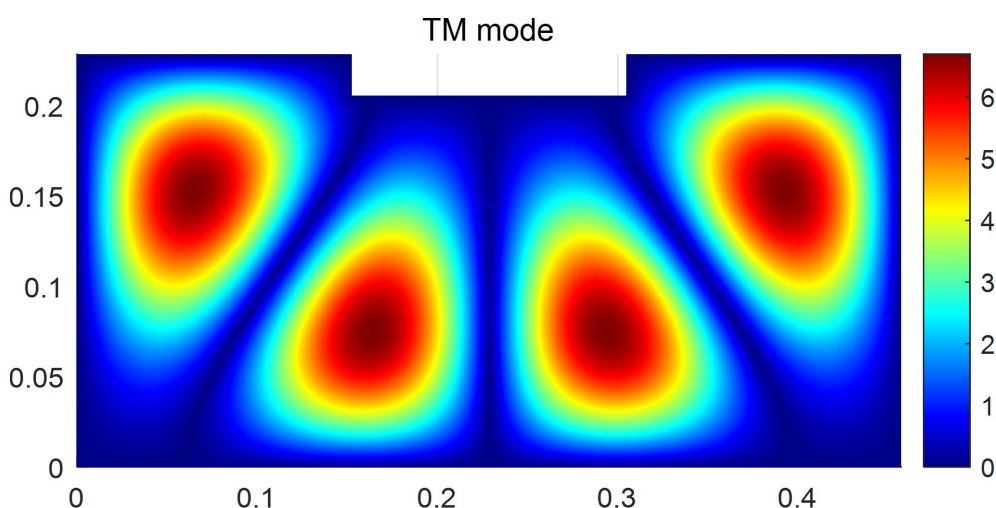


图 5-16 TM 第四高次模

将图 5-7——图 5-16 与矩形空波导场分布图

TE: 图 4-14——图 4-18

TM: 图 4-24——图 4-28

相对比，可以发现该不规则空波导的场分布，无论是 TE 模式还是 TM 模式均与矩形空波导具有类似的地方，性状相似度较高，离被扣掉处较近的部分变化较大，距离较远的部分变化较小。在绘图之前得到的猜想得以验证，即：该不规则波导的前 5 个本征模的场分布也与矩形波导的前 5 个本征模的场分布类似。

至此，实验所编写的程序经过了空波导理论值的检验，并成功利用该程序高效、准确的求出了一不规则的波导的前 5 个本征模的传播常数，并绘制出了场分布图像，实验全过程完成。

六、实验总结

通过本次实验，我深入学习和掌握了有限元方法和 **matlab** 的使用方法，并利用 **matlab** 求出了矩形空波导本征值问题的解。

杨老师刚讲这个问题时，我是一头雾水的。之前的一维问题有限元求解较为简单，没想到升级成二维之后，复杂度直线上升。

比如在一维问题中，对区域的离散只需要将求解区域分成 n 段即可。而二维问题是一个平面，需要对平面分成很多个小块，就让人感觉到问题难度的上升。在老师讲这个问题的时候，我的脑子就已经在想程序该怎么去写，才能实现这个问题的求解，让我十分苦恼的事情就是在开头区域离散就不会了，如果是分割成正方形的话，还好分割，而老师讲的是用三角形进行分割，一下子给我整不会了，绞尽脑汁的想这玩意怎么写，无果，遂等着老师讲。当然问题不止这一个。即使把划分给我，让我去写程序，去组装矩阵，也是不知道从哪里入手为好。

直到老师在最后讲了 **pde** 工具，才把我这些问题解决。我至今还能记得住杨老师那句话：“要是让咱自己去剖分，那就太复杂了。”随后杨老师讲了 **pde** 工具的使用方法，这时候我也感受到了 **matlab** 的强大。

划分出来了，第二个问题还没有解决，就是程序从哪里入手。这个问题也让 **pde** 工具解决了。在划分过后，**pde** 工具能够导出 **p**、**e**、**t** 矩阵，这三个矩阵就包含了该网格划分的全部信息：点的坐标、每个小三角形对应的三个点的全局编号、边界上的点的全局编号。有了这些矩阵，我的编程基本思路就出来了，可以说，开头最难的部分已经用 **pde** 工具解决了，剩下需要自己实现的过程就是根据 **ppt** 上的公式把程序编写出来即可，较为简单。

下课后当天晚上，我就照着 **ppt** 上的公式，把程序写出来了。写的过程也遇到了很多问题，最终还是解决了。写的过程感觉很难，但是写出来后再回头去看，也并没有那么复杂。然后我就开开心心的算了算理论值，并将程序结果与理论值做对比，发现不对。回头看了看程序，有一个地方有点小问题，改正后，与理论值相符，顿觉任务已经完成。

没想到的是，当我重启电脑后，再次运行程序，结果和理论值对不上了，这一下子给我干蒙了，我什么都没动，为什么同样的程序，同样的划分，却得到不一样的结果？

发现问题，就要解决问题。我反复去查找程序中的问题，去用不同的划分运行，去对比结果，神奇的是有时候能和理论值对上，有时候对不上，而且对不上

的时候多，当天调试程序到凌晨四点多，实在是扛不住了，遂睡觉，准备第二天上课问老师。

第二天上课，我找到杨老师并说明了我一晚上离奇的遭遇，老师很耐心的看了我的结果，并打开电脑让我看了看理论值，我发现我的答案其实是对的，是我把理论值算错了，我是用 excel 算的理论值，可能中间公式写错了。这时杨老师让我用 matlab 算理论值，如醍醐灌顶，看来是当天晚上已经神志不清了，居然没想到 matlab 可以很简单的算出理论值。

上课期间，杨老师不断的检查我的程序，帮我寻找问题，给出指导意见，并提供实用、高效的排查方法，对我成功寻找到引起问题的原因起到了重要作用。最终，在老师的指导下，我成功寻找到了问题：

（1）理论值计算错误

引起这个问题的原因很搞笑，波导的高度应该是 0.2286，而我在计算的时候用的数据是 0.2886，当我改成正确的数据后，用 excel 计算出的理论值和用 matlab 计算出的理论值是一样的。

（2）程序运行没有清理工作区矩阵

这个问题是引起为什么我的程序经常输出一些不正确的特征值，以及每次运行结果都不一样的原因。由于工作区的 p、e、t 矩阵在运行完没有被清理掉，在导入新的网格 p、e、t 矩阵时，如果新的划分更密集，新的 p、e、t 矩阵会完全覆盖掉旧的，这时程序运行结果是正确的。但是如果新的网格划分比旧的划分稀疏，新的 p、e、t 矩阵的大小比原来的 p、e、t 矩阵要小，不可能完全覆盖，那这时候工作区 p、e、t 前半部分的数据是新的划分，后半部分是没有被覆盖掉的，旧网格的数据，即 p、e、t 矩阵的内容是错误的，自然运行程序得到的结果是错误的。这个问题也是较难发现的。

当找到出现问题的原因，才算完整的完成了实验。总结来看，所有问题的出现都不在程序上，程序是正确的。而引起问题的，甚至让我熬夜到 4 点的，居然是自己粗心造成的小问题。这也让我深刻感受到了一个人生道理：任何一个小的错误，都有可能使得前功尽弃，甚至为此付出沉重的代价。

本次实验的成功完成，离不开杨老师的耐心指导、讲解。尤其是在我陷入迷茫，完全不知问题所在时，杨老师的帮助给我指明了方向，成功引导我找到了问题。在此对杨老师的付出表示诚挚的感谢，老师辛苦了！